

1. УВОД И ПОСТАВКА ЗАДАТКА

1.1. Циљ задатка

Циљ овог задатка јесте пројектовање, рачунарска симулација и анализа рада дигиталних П (Пропорционалног) и ПИ (Пропорционално-Интегралног) регулатора у систему са затвореном повратном спрегом. Управљани објекат представља једносмерни електромотор (односно процес првог реда) дефинисан континуалном преносном функцијом првог реда која износи:

$$G(s) = \frac{K}{\tau s + 1} = \frac{0.8}{0.3s + 1}$$

Где је $K = 0.8$ статичко појачање објекта, а $\tau = 0.3$ [s] временска константа система. Кроз практичну имплементацију у окружењу Siemens TIA Portal (SCL) и визуелизацију на HMI Trend View графику, циљ је верификовати следеће објективне показатеље квалитета управљања:

- Поређење П и ПИ режима: Анализа утицаја пропорционалног појачања K_s на брзину одзива и доказивање присуства статичке грешке у стационарном стању при примени само П регулатора.
- Елиминација статичке грешке: Показивање како увођење интегралног дејства (K_i) потискује стационарну грешку на нулу.
- Спречавање засићења интегратора (Anti-Reset Windup): Практична верификација алгоритма за ограничење интегралне суме у условима засићења (односно сатурације) управљачког сигнала.

1.2. Одређивање почетних параметара и дискретизација

Да би се континуални алгоритам могао реализовао у PLC управљачком блоку OB30 (Cyclic Interrupt) са периодом одабирања $T = 0.04$ [s] (односно 40 [ms]), извршена је дискретизација објекта и прорачун параметара регулатора.

Применом Ојлерове методе (или аналитичким решењем диференцијалне једначине за период $T = 0.04$ [s] и $\tau = 0.3$ [s]), добија се релација којом се у SCL коду у реалном времену израчунава излаз објекта $y[k]$, на следећи начин:

$$e^{-\frac{T}{\tau}} = e^{-\frac{0.04}{0.3}} = e^{-0.13333} \approx 0.87517$$

$$K \cdot (1 - e^{-\frac{T}{\tau}}) = 0.8 \cdot (1 - 0.87517) = 0.8 \cdot 0.12483 \approx 0.09986$$

$$y[k] = e^{-\frac{T}{\tau}} \cdot y[k-1] + K \left(1 - e^{-\frac{T}{\tau}}\right) \cdot u[k]$$

$$y[k] = 0.8751733 \cdot y[k-1] + 0.0998613 \cdot u[k-1]$$

Знајући ово, могуће је касније модификовати модел електромотора мењајући вредности K и τ , чиме бисмо успорили или убрзали систем, дајући нам другачије облике криве на графику, са обзиром на промену брзине система. Подешавање параметара ПИ регулатора спроведено је методом поништавања доминантног пола објекта (pole-zero cancellation), где се интегрална временска константа регулатора T_i изједначава са временском константом објекта τ , односно:

$$T_i = \tau = 0.3 \text{ s}$$

На основу релације за континуално интегрално појачање $K_i = K_c/T_i$ и за усвојено пропорционално појачање у ПИ режиму $K_c = 3$, израчунава се континуално интегрално појачање:

$$K_i = \frac{K_c}{\tau} = \frac{3.0}{0.3 \text{ s}} = 10.0 \text{ s}^{-1}$$

Пошто се у дискретном SCL коду интегрална сума ажурира у сваком кораку одабирања $T = 0.04 \text{ [s]}$, у коду се рачуна дискретни корак интеграције K_{i_step} :

$$K_{i_step} = K_i \cdot T = 10.0 \cdot 0.04 = \mathbf{0.4}$$

Са одређеним почетним параметрима можемо да кренемо на израду самог SCL кода у TIA Portal-у.

Будући да је овај рад заправо наставак задатка 1, као коришћени контролер се поново бира Simatic S7-1200 CPU, и то тачније модел CPU 1214C DC/DC/DC, верзија 4.4.

2. ДЕФИНИСАЊЕ И АНАЛИЗА PLC ПРОМЕНЉИВИХ (PLC TAGS)

За исправно функционисање математичког модела, алгоритама управљања и комуникације са HMI панелом, дефинисана је табела глобалних PLC променљивих (Default tag table). Све променљиве су Real типа (осим управљачких битова) како би се омогућила обављање прецизних прорачуна са добијеним вредностима током симулације.

Комплетирана табела PLC тагова треба да изгледа као следећа табела:

Default tag table									
	Name	Data type	Address	Retain	Acces...	Writa...	Visibl...	Cc	
1	 vrednost_upravljanja	Real	 %MD10		☐	☒	☒	☒	
2	 izlazna_vrednost_objekta	Real	%MD14	☐	☒	☒	☒	☒	
3	 matematicko_upravljanje	Real	%MD18	☐	☒	☒	☒	☒	
4	 stacionarna_greska	Real	%MD30	☐	☒	☒	☒	☒	
5	 vrednost_integrala	Real	%MD22	☐	☒	☒	☒	☒	
6	 izbor_rezima_regulatora	Bool	%M0.1	☒	☒	☒	☒	☒	
7	 mod_motora	Bool	%M0.0	☒	☒	☒	☒	☒	
8	 proporcionalno_dejstvo	Real	%MD26	☐	☒	☒	☒	☒	
9	 Kc_P	Real	%MD34	☐	☒	☒	☒	☒	
10	 Kc_PI	Real	%MD38	☐	☒	☒	☒	☒	
11	 Ki_step	Real	%MD42	☐	☒	☒	☒	☒	
12	 izbor_zadatka	Bool	%M0.2	☒	☒	☒	☒	☒	
13	 Kc_HMI_input	Real	%MD46	☐	☒	☒	☒	☒	
14	 Ki_HMI_input	Real	%MD50	☐	☒	☒	☒	☒	
15	<Add new>			☐	☒	☒	☒	☒	

“vrednost_upravljanja”: Задата вредност (w) коју корисник уноси преко HMI-ја (жељена вредност излаза).

“izlazna_vrednost_objekta”: Тренутна излазна вредност из симулатора објекта (y) која се приказује на HMI графику.

“matematicko_upravljanje”: Укупно израчунати управљачки сигнал (u) који регулатор шаље објекту управљања.

“stacionarna_greska”: Разлика између задате и тренутне вредности ($e = w - y$).

“vrednost_integrala”: Акумулирана вредност интегралног дејства (u_i) унутар ПИ регулатора.

“izbor_rezima_regulatora”: Дугме на HMI-ју за избор режима рада регулатора (FALSE = П режим, TRUE = ПИ режим).

“mod_motora”: Главни прекидач на HMI-ју за укључивање/исључивање рада система (START/STOP дугме).

“proporcionalno_dejstvo”: Израчуната вредност пропорционалне компоненте управљања ($u_p = K_c * e$).

“Kc_P”: Пропорционално појачање које се примењује када је регулатор у П режиму.

“Kc_PI”: Пропорционално појачање које се примењује када је регулатор у ПИ режиму.

“Ki_step”: Израчунати дискретни корак интеграције ($K_{i_step} = K_i * T_s$) за период одабирања од 40 [ms].

“izbor_zadatka”: Дугме на HMI-ју за избор структуре управљања (FALSE = затворена повратна спрега, TRUE = отворена петља).

“Kc_HMI_input”: Поље на HMI-ју преко којег оператер задаје пропорционално појачање Kc.

“Ki_HMI_input”: Улазно поље на HMI-ју преко којег оператер задаје интегрално појачање Ki.

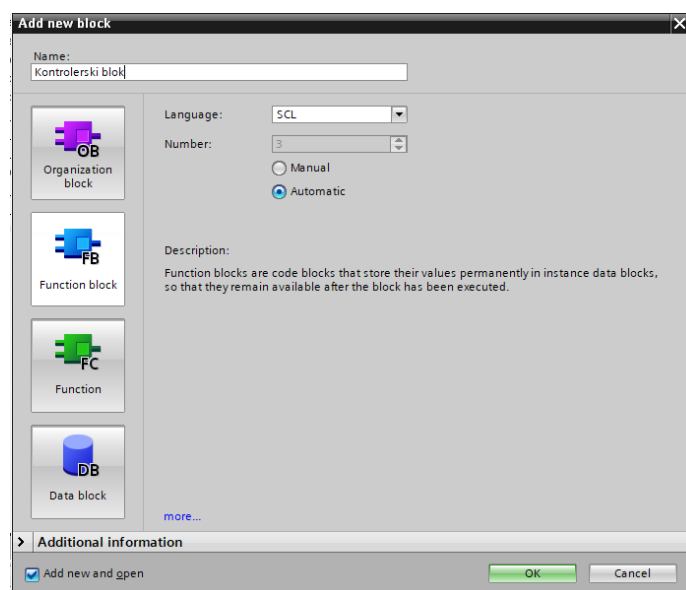
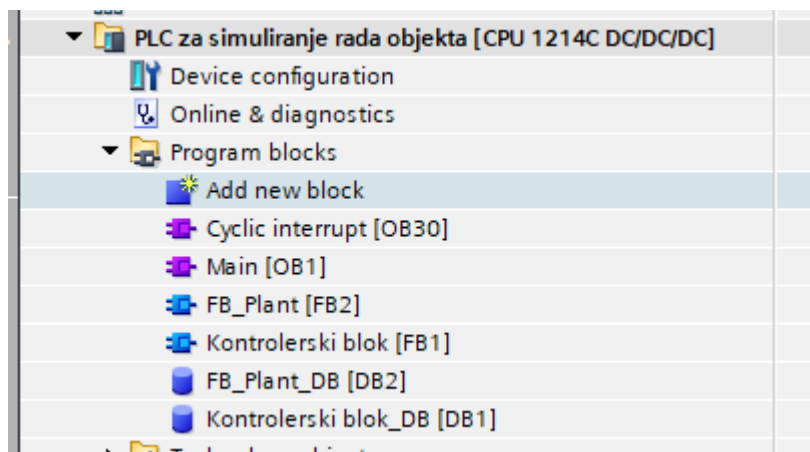
Како би детаљно објаснили улоге променљивих, можемо их сврстати у четири групе:

- Управљачки битови (mod_motora, izbor_rezima_regulatora, izbor_zadatka), служе за избор режима рада и управљање симулацијом. mod_motora омогућава ресетовање свих променљивих на 0 када је мотор искључен, izbor_rezima_regulatora преусмерава извршавање SCL кода између П и ПИ алгоритма, док izbor_zadatka омогућава избор између затворене повратне спреге и отворене петље.
- Улазне и излазне процесне променљиве (vrednost_upravljanja, izlazna_vrednost_objekta, stacionarna_greska), чине основну петљу са повратном спрегом. У сваком циклусу од 40 [ms], сигнал грешке stacionarna_greska се прерачунава као разлика задате вредности vrednost_upravljanja и тренутног одзива мотора izlazna_vrednost_objekta.
- Компоненте управљања (proporcionalno_dejstvo, vrednost_integrала, matematicko_upravljanje), proporcionalno_dejstvo представља тренутно дејство сразмерно грешци, док vrednost_integrала акумулира претходна одступања ради елиминације статичке грешке. matematicko_upravljanje формира коначни сигнал који делује на дискретни модел мотора.
- Параметри регулатора (Kc_P, Kc_PI, Ki_step, Kc_HMI_input, Ki_HMI_input), нам омогућавају независно подешавање појачања за различите режиме рада. Раздвајањем Kc за П и ПИ режим омогућено нам је да посматрамо пропорционално појачање у случају деловања обе врсте регулатора.

3. ИМПЛЕМЕНТАЦИЈА У СИСТЕМУ TIA PORTAL (SCL)

3.1. Креирање и програмирање контролерског блока

Након креирања табеле глобалних тагова (Default tag table), приступа се изради првог кључног дела пројекта– контролерског блока (FB1). Овај блок је реализован као функцијски блок (Function Block – FB) у SCL језику, чиме је омогућена његова потпуна реализованост и независност од остатка кода. Па тако:



Пре писања саме управљачке логике, дефинисан је интерфејс блока који се састоји од улазних, излазних и статичких променљивих које се уносе на следећи начин:

Kontrolerski blok									
	Name	Data type	Default value	Retain	Accessible f...	Writa...	Visible in ...	Setpoint	
1	Input								
2	enable	Bool	false	Non-ret...	☒	☒	☒	☐	
3	open_loop	Bool	false	Non-retain	☒	☒	☒	☐	
4	pi_mode	Bool	false	Non-retain	☒	☒	☒	☐	
5	w_setpoint	Real	0.0	Non-retain	☒	☒	☒	☐	
6	y_actual	Real	0.0	Non-retain	☒	☒	☒	☐	
7	Kc_in	Real	0.0	Non-retain	☒	☒	☒	☐	
8	Ki_in	Real	0.0	Non-retain	☒	☒	☒	☐	
9	Output								
10	u_out	Real	0.0	Non-retain	☒	☒	☒	☐	
11	e_out	Real	0.0	Non-retain	☒	☒	☒	☐	
12	u_p_out	Real	0.0	Non-retain	☒	☒	☒	☐	
13	u_i_out	Real	0.0	Non-retain	☒	☒	☒	☐	
14	Kc_P_out	Real	0.0	Non-retain	☒	☒	☒	☐	
15	Kc_PI_out	Real	0.0	Non-retain	☒	☒	☒	☐	
16	Ki_step_out	Real	0.0	Non-retain	☒	☒	☒	☐	
17	InOut								
18	<Add new>								
19	Static								
20	u_integral_stat	Real	0.0	Non-retain	☒	☒	☒	☐	
21	Tempo								

Улазни параметри (под групом Input):

- enable (Bool): Дозвола рада регулатора (активира се преко тага mod_motora).
- open_loop (Bool): Избор структуре управљања (FALSE = затворена повратна спрега, TRUE = отворена петља).
- pi_mode (Bool): Избор режима рада регулатора (FALSE = П режим, TRUE = ПИ режим).
- w_setpoint (Real): Задата вредност (референтни сигнал са HMI-ја).
- y_actual (Real): Тренутни одзив/излаз објекта.
- Kc_in (Real): Пропорционално појачање унето са HMI-ја (Kc_HMI_input).
- Ki_in (Real): Интегрално појачање унето са HMI-ја (Ki_HMI_input).

Излазни параметри (под групом Output):

- u_out (Real): Укупно израчунати управљачки сигнал који се прослеђује објекту.
- e_out (Real): Тренутна статичка грешка одступања ($e = w - y$).
- u_p_out (Real): Пропорционална компонента управљачког сигнала.
- u_i_out (Real): Интегрална компонента управљачког сигнала.
- Kc_P_out (Real): Вредност појачања K_c за П режим.
- Kc_PI_out (Real): Вредност појачања K_c за ПИ режим.
- Ki_step_out (Real): Израчунати дискретни корак интеграције (K_{i_step}).

Статичке променљиве (под групом Static):

- stat_integral (Real): Интерни акумулатор интегралне суме који чува вредност између два циклуса одабирања.

Алгоритам унутар контролерског блока извршава следеће операције:

1. Провера дозволе рада (enable), Ако је enable = FALSE, све излазне променљиве и интерна сума stat_integral ресетују се на 0.0.
2. Прорачун грешке и дискретног корака, грешка одступања рачуна се као $e = w - y$ (уколико је изабрана затворена петља) односно $e = w$ (уколико је изабрана отворена петља).
3. Дискретни корак интеграције ажурира се на основу унетог интегралног појачања: $K_{i_step} = K_i * T_s = K_i * 0.04$.
4. Формирање управљања (П или ПИ режим), за П режим (pi_mode = FALSE) рачуна се само пропорционално дејство $u_p = K_c * e$, док се stat_integral ресетује на 0.0. За ПИ режим (pi_mode = TRUE) акумулира се интегрална сума а укупни сигнал формира се као збир пропорционалног и интегралног дејства ($u = u_p + u_i$).

Сам извршни SCL код се може реализовати на следећи начин:

```

konacan_rad_supervisor ▸ PLC za simuliranje rada objekta [CPU 1214C DC/DC/DC] ▸ Program blocks ▸ Kontrolerski blok [FB1]

IF... CASE... FOR... WHILE... DO... (*...*) REGION
1 IF NOT #enable THEN
2   #e_out := 0.0;
3   #u_p_out := 0.0;
4   #u_integral_stat := 0.0;
5   #u_i_out := 0.0;
6   #u_out := 0.0;
7 ELSE
8   IF #open_loop THEN
9     #e_out := 0.0;
10    #u_p_out := 0.0;
11    #u_integral_stat := 0.0;
12    #u_i_out := 0.0;
13    #u_out := #w_setpoint;
14
15    ELIF NOT #pi_mode THEN
16      #Kc_P_out := #Kc_in;
17      #e_out := #w_setpoint - #y_actual;
18      #u_p_out := #Kc_P_out * #e_out;
19      #u_integral_stat := 0.0;
20      #u_i_out := 0.0;
21      #u_out := #u_p_out;
22
23    ELSE
24      #Kc_PI_out := #Kc_in;
25      #Ki_step_out := #Ki_in * 0.04; // za Ts = 0.04 s
26
27      #e_out := #w_setpoint - #y_actual;
28      #u_p_out := #Kc_PI_out * #e_out;
29      #u_integral_stat := #u_integral_stat + (#Ki_step_out * #e_out);
30
31      // ARW mehanizam
32      IF #u_integral_stat > 125.0 THEN
33        #u_integral_stat := 125.0;
34      ELIF #u_integral_stat < 0.0 THEN
35        #u_integral_stat := 0.0;
36      END_IF;
37
38      #u_i_out := #u_integral_stat;
39      #u_out := #u_p_out + #u_i_out;
40    END_IF;
41
42    // Saturacija regulatora
43    IF #u_out > 125.0 THEN
44      #u_out := 125.0;
45    ELIF #u_out < 0.0 THEN
46      #u_out := 0.0;
47    END_IF;
48  END_IF;

```

Након извршеног писања кода, програмски блок обавезно треба компајлирати и проверити да ли се јављају евентуалне грешке. Ако се након компајлирања блока не јаве грешке можемо прећи на следећи програмски блок.

3.2. Креирање и програмирање симулатора објекта

Након успешног дефинисања и програмирања контролерског блока, следећи корак је израда засебног функцијског блока Simulator објекта, или у овом пројекту FB_Plant (FB2). Овај блок садржи дискретни математички модел објекта управљања (електромотора), чиме се симулира његов динамички одзив у реалном времену на основу управљачког сигнала добијеног из контролерског блока.

Након убацивања новог функцијског блока FB, Поново је потребно дефинисати интерфејс блока, на следећи начин:

FB_Plant									
	Name	Data type	Default value	Retain	Accessible f...	Writa...	Visible in ...	Setpoint	Cor
1	Input								
2	enable	Bool	false	Non-ret...	☒	☒	☒	☐	
3	u_in	Real	0.0	Non-retain	☒	☒	☒	☐	
4	Output								
5	y_out	Real	0.0	Non-retain	☒	☒	☒	☐	
6	InOut				☐	☐	☐	☐	
7	<Add new>				☐	☐	☐	☐	
8	Static				☐	☐	☐	☐	
9	u_stat	Real	0.0	Non-retain	☒	☒	☒	☐	
10	y_stat	Real	0.0	Non-retain	☒	☒	☒	☐	
11	Temp				☐	☐	☐	☐	

Улазни параметри (под групом Input):

- enable (Bool): Дозвола рада симулатора (повезано са тагом mod_motora).
- u_in (Real): Управљачки сигнал добијен из регулатора (matematicko_upravljanje).

Изразни параметри (под групом Output):

- y_out (Real): Тренутни израчунати излаз/одзив објекта (izlazna_vrednost_objekta).

Статичке променљиве (под групом Static):

- y_stat (Real): Статичка променљива која чува вредност излаза из претходног корака одабирања ($y[k-1]$).
- u_stat (Real): Статичка променљива која чува вредност управљачке променљиве из претходног корака одабирања ($u[k-1]$).

Алгоритам унутар FB_Plant блока извршава следећа израчунавања:

1. Провера дозволе рада (enable): Уколико је enable = FALSE, излазна вредност y_out и претходно стање y_stat одмах се ресетују на 0.0.
2. Дискретни прорачун: Када је рад омогућен, извршава се дискретна диференцијална једначина за период одабирања $T_s = 40$ [ms] и временску константу $\tau = 0.3$ [s]:

$$y[k] = 0.8751733 \cdot y[k - 1] + 0.0998613 \cdot u[k - 1]$$

Након израчунавања новог одзива, вредност y_{out} се додељује статичкој променљивој y_{stat} , чиме се чува претходно стање за следећи такт одабирања (40 [ms]). Иста логика се примењује и за променљиву u_{out} , која се додељује статичкој променљивој u_{stat} .

Сам извршни SCL код се може реализовати на следећи начин:

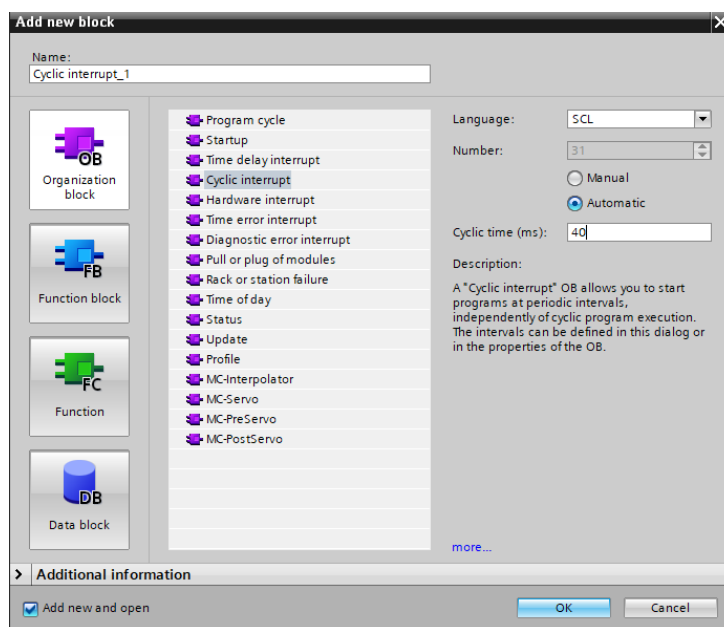
```

1 IF NOT #enable THEN
2   #y_stat := 0.0;
3   #y_out := 0.0;
4   #u_stat := 0.0;
5 ELSE
6   // Diskretni model objekta prvog reda: y(k) = 0.87517 * y(k-1) + 0.09986 * u(k-1)
7   #y_stat := (0.87517 * #y_stat) + (0.09986 * #u_stat);
8   #y_out := #y_stat;
9   #u_stat := #u_in;
10 END_IF;
```

Након извршеног писања кода, програмски блок обавезно треба компајлирати и проверити да ли се јављају евентуалне грешке. Ако се након компајлирања блока не јаве грешке можемо прећи на следећи програмски блок.

3.3. Инстанцирање и повезивање у OB30 (Cyclic interrupt)

Да би се дискретни алгоритми управљања и симулације извршавали у тачно дефинисаним временским интервалима, њихово позивање се не врши у основном циклусу програма (OB1), већ унутар организационог блока са цикличним прекидом који додајемо у задатак OB30 (Cyclic interrupt).



Пре писања кода, при убацивању OB30 блока подешен је параметар одабирања Cyclic time који је постављен на 40 [ms]. Овим се гарантује да PLC прекида извршавање главног програма на сваких 40 [ms] и позива OB30, чиме су физичке временске константе $T_s = 0.04$ [s] употребљене у SCL коду, и у потпуности усклађене са реалним временом извршавања.

Унутар OB30 блока креиране су две засебне инстанце функцијских блокова са својим припадајућим позивним блоковима (Data Blocks):

- Kontrolerski blok_DB (DB1): Инстанца за регулаторски блок.
- FB_Plant_DB (DB2): Инстанца за симулатор објекта.

Повезивањем улаза и излаза ових блокова са глобалним таговима из Default tag table-а формирана је комплетна дигитална управљачка петља са повратном спрегом.

Формирање Data blocks се најједноставније ради превлачењем самих блокова који се позивају из самог Project tree-а директно у место за SCL код OB30 блока.

Након убацивања блокова, и формирања одговарајућих дата блокова, може се прећи на следећи корак. Крајњи OB30 програмски блок треба изгледати на следећи начин:

Испитни задатак из Аутоматског управљања

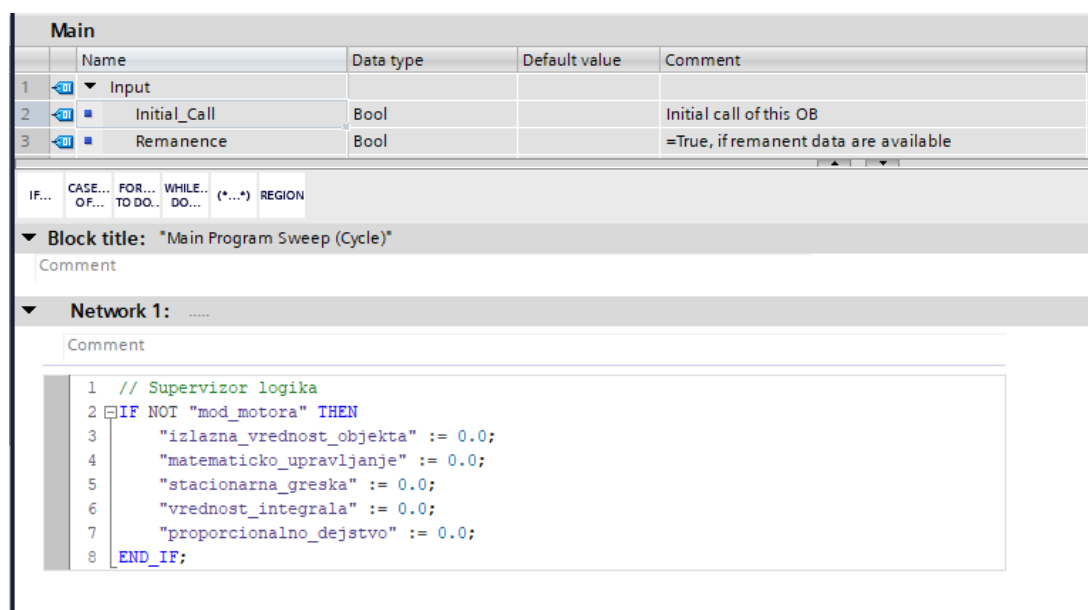
```
1 □ "Kontrolerski blok_DB"(enable:="mod_motora",
2     open_loop:="izbor_zadatka",
3     pi_mode:="izbor_rezima_regulatora",
4     w_setpoint:="vrednost_upravljanja",
5     y_actual:="izlazna_vrednost_objekta",
6     Kc_in:="Kc_HMI_input",
7     Ki_in:="Ki_HMI_input",
8     u_out=>"matematicko_upravljanje",
9     e_out=>"stacionarna_greska",
10    u_p_out=>"proporcionalno_dejstvo",
11    u_i_out=>"vrednost_integrала",
12    Kc_P_out=>"Kc_P",
13    Kc_PI_out=>"Kc_PI",
14    Ki_step_out=>"Ki_step");
15 □ "FB_Plant_DB"(enable := "mod_motora",
16     u_in := "matematicko_upravljanje",
17     y_out => "izlazna_vrednost_objekta");
18
```

3.4. Надзорна логика у OB1 (Main)

Иако се основна контролна петља заснована на прекидима извршава унутар OB30, главни организациони блок OB1 (Main) задржава надзорну улогу над целим системом (такозвани супервизор). OB1 се извршава континуирано у оквиру слободног циклуса PLC-а (Cyclic program execution) и користи се за обезбеђивање безбедносних стања и ресетовање свих процесних променљивих када систем није активан.

Унутар OB1 блока имплементирана је логика која проверава статус главног прекидача `mod_motora`, провера искљученог стања (`mod_motora = FALSE`), односно уколико је мотор искључен, надзорна логика форсира ресетовање свих глобалних процесних тагова на нулу. Тиме се спречава задржавање старих вредности на HMI графику, чувању неважећих интегралних сума или слању било каквог управљачког сигнала ка излазима.

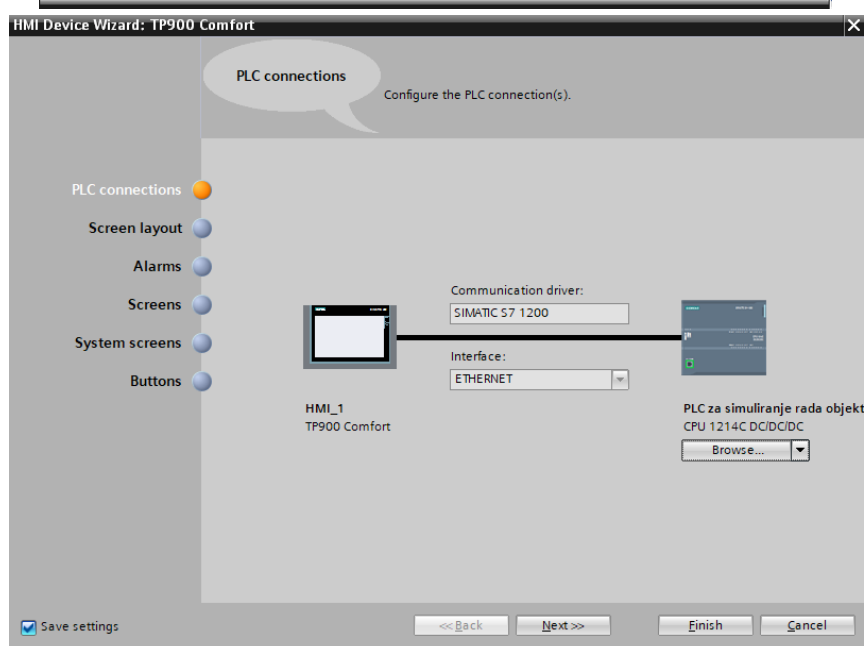
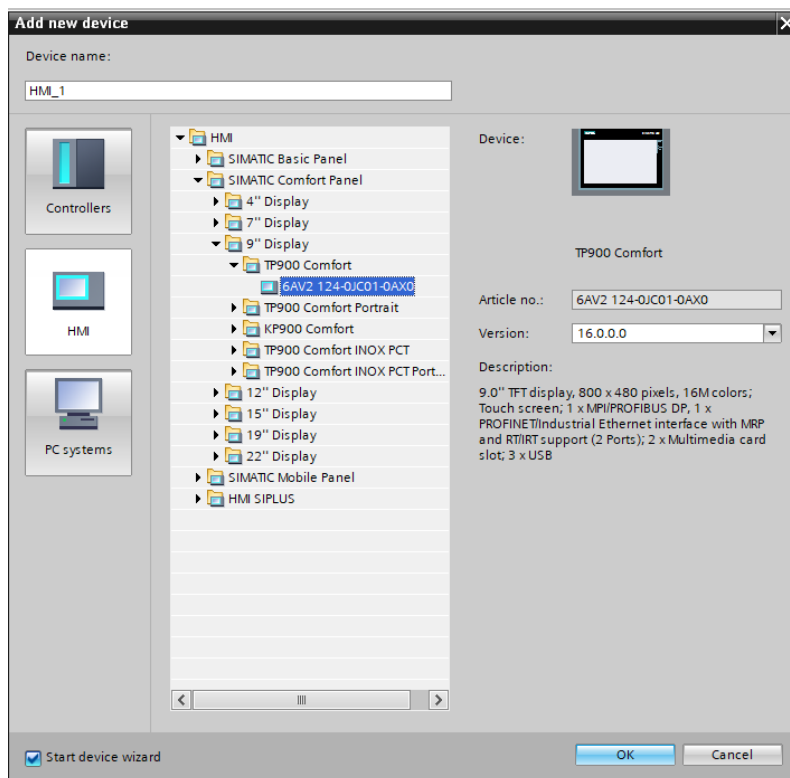
Провера активног стања (`mod_motora = TRUE`). Када је мотор укључен, OB1 препушта потпуну контролу над ажурирањем променљивих OB30 блоку и не врши никакве измене на процесним променљивама.



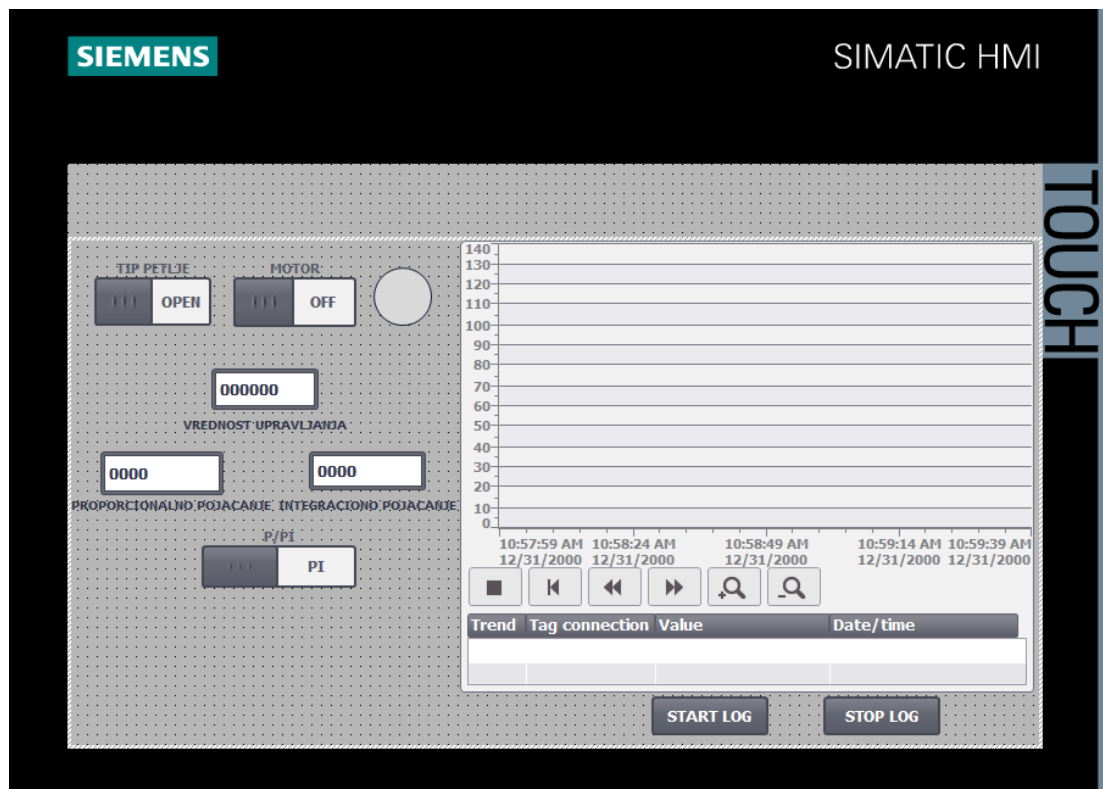
Након комплетирања кода, потребно је поново целокупно компајлирати све промене у PLC-у, и затим након евентуалних преправљања грешака приступити изради следећег корака који јесте убацивање и едитовање HMI панела за читавање вредности.

4. КРЕИРАЊЕ И КОНФИГУРАЦИЈА HMI ЕКРАНА

За потребе визуелизације и директног управљања процесом у реалном времену, потребно је да направимо HMI екран (Root screen) у оквиру WinCC Runtime/Basic HMI окружења. За сврху овог задатка одабран је KTP 900 Comfort PN, чија се имплементација у TIA portal врши одабиром функције Add new device са врха Project tree-ја. Након избора панела, треба да се одабере наш PLC са падајуће листе чиме се остварује конекција између ова два дела хардвера.



Панел који правимо састоји се из више целина:



Главни командни панел се састоји од управљачких елемената:

- Дугме за START / STOP (mod_motora), дозвољава или блокира рад целе симулације и свих управљачких блокова, као и лампица која мења боју у зависности од тога у ком је режиму мотор.
- Прекидач P / PI (izbor_rezima_regulatora), омогућава кориснику тренутну промену алгорита регулатора са пропорционалног на пропорционално-интегрални.
- Прекидач Zatvorena / Otvorena petlja (izbor_zadatka), служи за избор између управљања у затвореној повратној спрези и рада у отвореној петљи.

Улазна поља за параметризацију (I/O Fields):

- Вредност управљања (vrednost_upravljanja), параметарско улазно поље за задавање жељене вредности излаза.
- Пропорционално појачање (Kc_HMI_input), односно поље за унос вредности пропорционалног појачања K_c .
- Интеграционо појачање (Ki_HMI_input), поље за унос вредности интегралног појачања K_i .

Централни део екрана заузима Trend View објекат који графички у реалном временом приказује временски одзив система ($y(t)$) у односу на задату вредност ($w(t)$).

Да би смо себи олакшали унос променљивих, додајемо у Default tag table HMI-а следеће тагове:

Default tag table						
Name	Data type	Connection	PLC name	PLC tag	Address	
izbor_rezima_regulatora	Bool	HMI_Conne...	PLC za simuliranje rada objekta	izbor_rezima_regulat...		
izbor_zadatka	Bool	HMI_Connectio...	PLC za simuliranje rada objekta	izbor_zadatka		
izlazna_vrednost_objekta	Real	HMI_Connectio...	PLC za simuliranje rada objekta	izlazna_vrednost_objekta		
Kc_HMI_input	Real	HMI_Connectio...	PLC za simuliranje rada objekta	Kc_HMI_input		
Kc_P	Real	HMI_Connectio...	PLC za simuliranje rada objekta	Kc_P		
Ki_HMI_input	Real	HMI_Connectio...	PLC za simuliranje rada objekta	Ki_HMI_input		
mod_motora	Bool	HMI_Connectio...	PLC za simuliranje rada objekta	mod_motora		
vrednost_upravljanja	Real	HMI_Connectio...	PLC za simuliranje rada objekta	vrednost_upravljanja		
<Add new>						

Даље, како би могли да резултате симулације експортујемо ван TIA Portal-а, потребно је да извршимо logging променљивих, које се задају у делу пројектног стабла названог Historical data, на следећи начин:

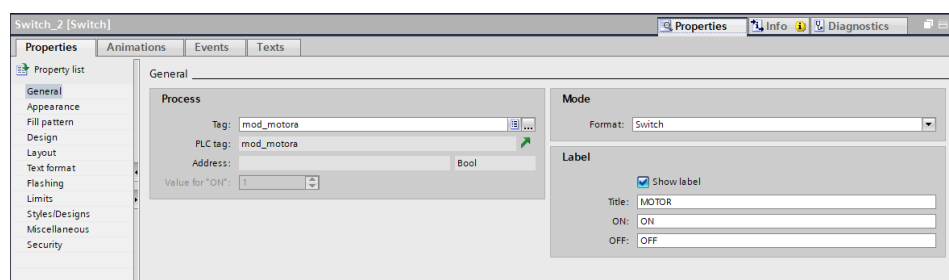
Data logs							
Name	Storage location	Data records	Path	Logging method	Number of sequen...	Fill level	Enable logging at runtime st
Logging_file	TXtfile (Unicode)	1000	IStorage Card SDLogs	Circular log	10	90	
<Add new>							

Logging tags						
Name	Process tag	Acquisition mode	Logging cycle	High limit	Low limit	Range for logging limits
Izlaz_objekta	izlazna_vrednost_objekta	On change				Within deadband
Upravljacka_promenljiva	vrednost_upravljanja	On change				Within deadband
<Add new>						

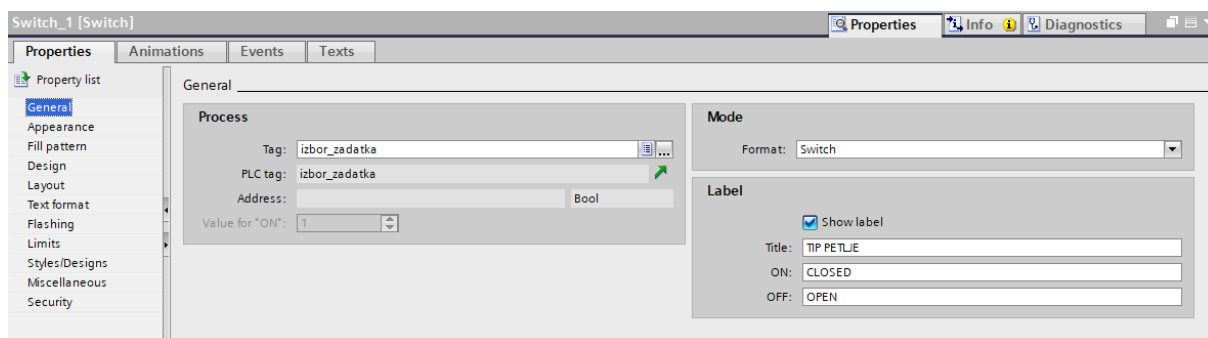
За потребе овог задатка вршиће се праћење само две променљиве, излазне вредности објекта, и вредност управљачке променљиве коју том систему задајемо. Важно је променити Acquisition mode да буде на On change, да би се вредности уписивале у текст фајл након сваке промене која им се деси.

Едитовање HMI панела се врши на делу Project tree-ја под називом Screens, па по дифолту Root Screen, или било који други скрин који додамо касније ако нам је потребан.

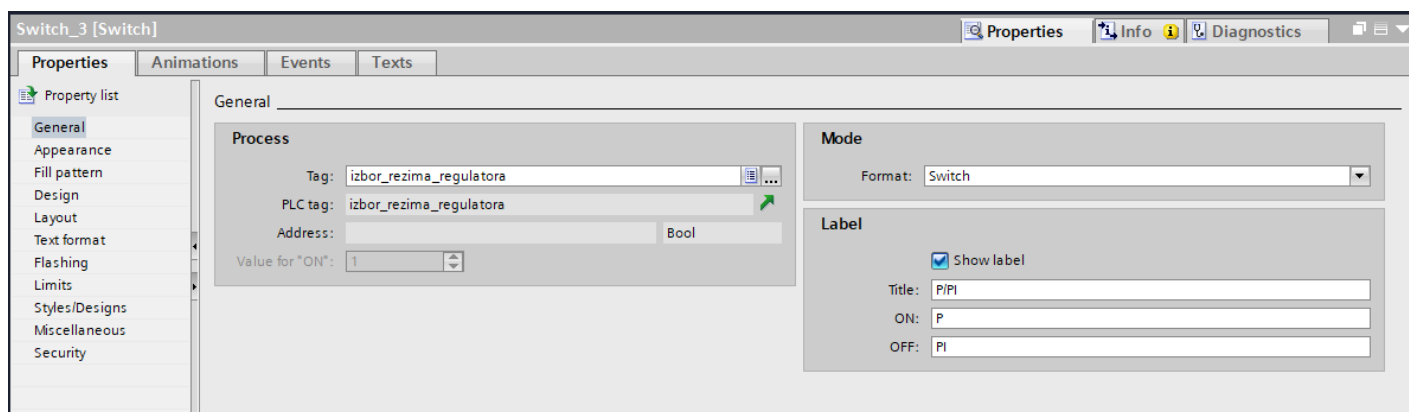
Ако кренемо од СТАРТ/СТОП прекидача, његова имплементација је прилично једноставна, из Toolbox-а са десне стране превучемо из секције Elements, прекидач односно Switch на панел. Кликом на тај прекидач и селектовањем Properties са доњег менија можемо едитовати понашање прекидача. Прекидач повезујемо са одговарајућим тагом који контролише дејство паљења односно гашења мотора.



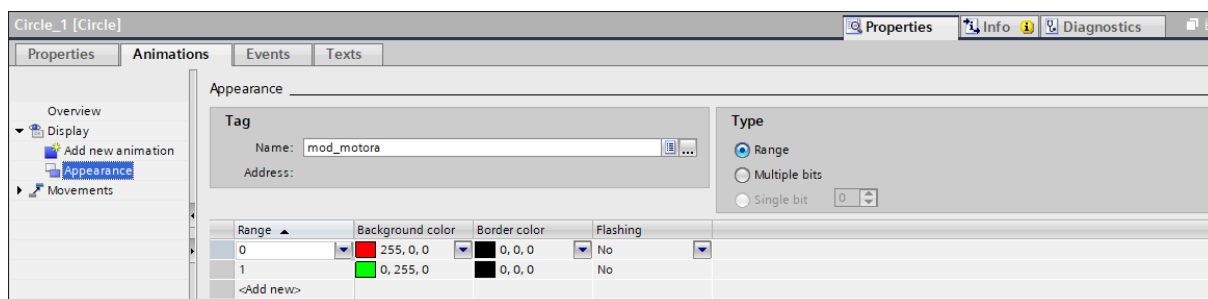
За прекидач који контролише пребацивање задатка из симулације објекта у отвореној петљи и задатка за симулацију затворене повратне спреге, користи се исти принцип као и за СТАРТ/СТОП прекидач, само се мења таг који се везује за функцију дугмета.



За прекидач који дефинише да ли је активан П или ПИ регулатор такође се користи иста логика, при чему се овај прекидач у потпуности искључује при пребацивању симулације у отворену спрегу, јер тада не постоји дејство регулатора, било то П или ПИ.



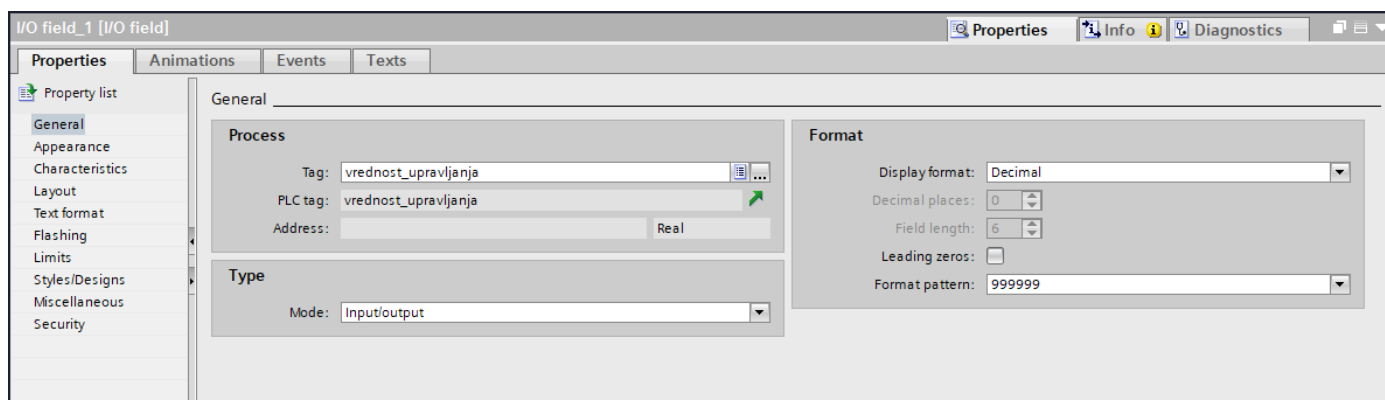
Како би убацили лампицу која мења боју у зависности од статуса мотора, и Toolbox-а на панел превлачимо из дела Basic objects – Circle. Пошто је статус мотора једноставна бинарна промена јединица и нула, у делу Animations од селектованог круга можемо изабрати Display -> Appearance и повезати понашање сијалице и статуса мотора поновно бирајући таг mod_motora, а затим испод тога дефинисати боје сијалице за одговарајуће модове мотора. У мом случају зелено за рад мотора, а црвено док је искључен.



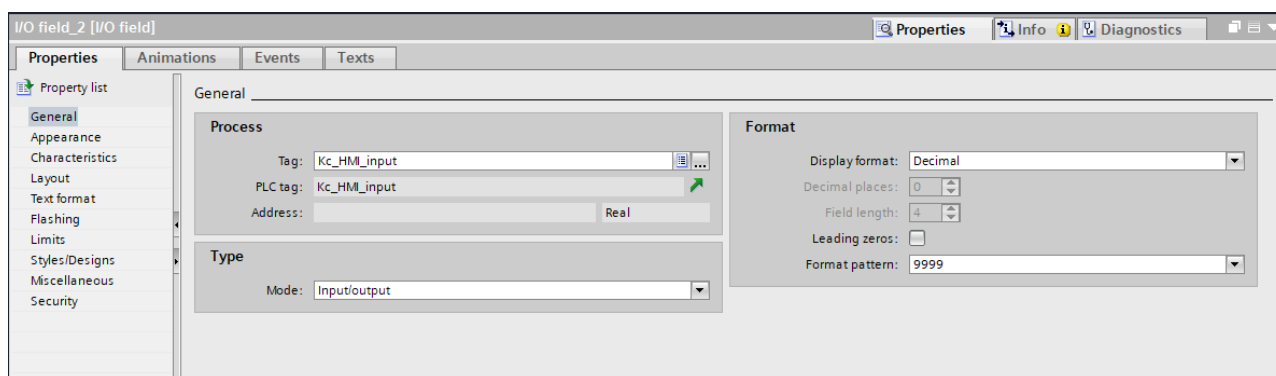
За остала поља у које можемо да уносимо вредност управљања која су нам потребна, из Toolbox-а са десне стране из одељка Elements бирамо I/O Field, и превлачимо га на наш панел. Затим у менију Properties ово поље повезујемо са одговарајућим тагом и дефинишемо децимални формат унешеног броја за свако поље које додајемо, односно:

Испитни задатак из Аутоматског управљања

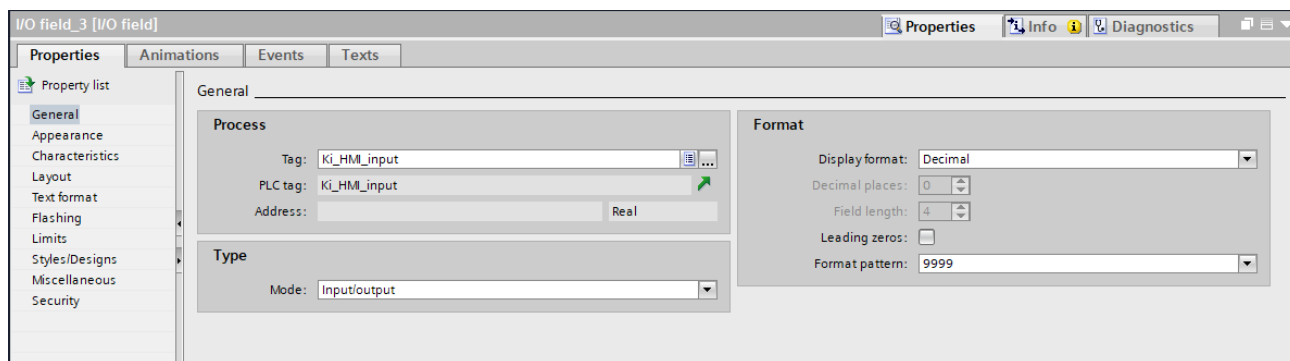
За вредност управљања:



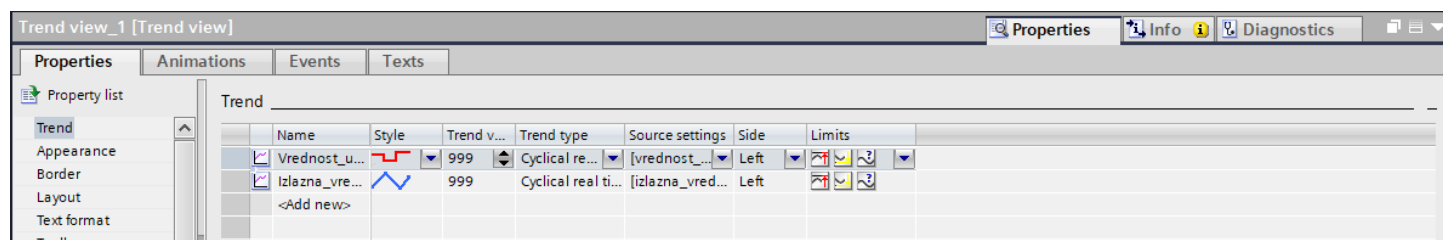
За вредност пропорционалног појачања:



Односно, за вредност интеграционог појачања регулатора:



За убацивање самог графика користимо из Toolbox-а елемент назван Trend View (или $f(x)$)
Trend View ако је коришћен Comfort панел при креирању HMI-а), на следећи начин:



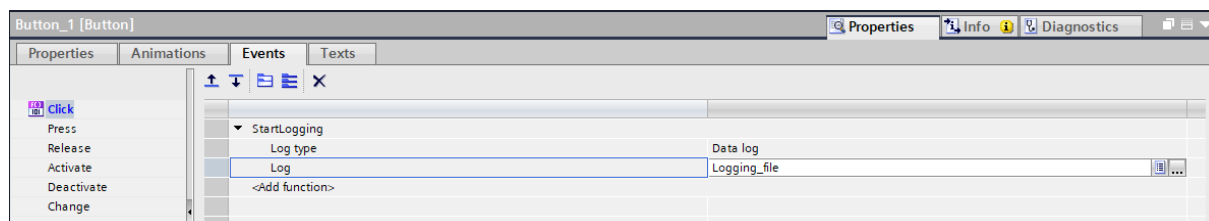
Потребно је подесити Trend type да буде Cyclical real time, како би пратио циклично извршавање кода на сваких 0,04 [s]. Убацујемо две линије, једна која представља сигнал контролера, а друга, у овом случају плава представља реалну брзину коју за тај сигнал постиже мотор, изражену у процентима.

За потребе додатне анализе одзива система и верификације управљачког алгоритма, на HMI екрану су имплементирају командни дугмићи за управљање евиденцијом података (Data Logging). Ови дугмићи омогућавају оператеру да ручно започне или заустави снимање процесних променљивих у реалном времену.

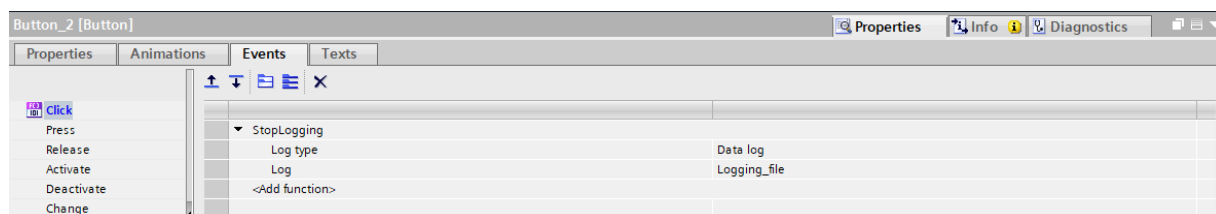
Активирањем ове функције, подаци се аутоматски записују у текстуалну датотеку у меморији контролера, што омогућава њихов каснији извоз и графичку обраду у спољним софтверским алатима (као што је MATLAB у нашем случају) ради упоређивања реалног одзива са симулационим моделима.

Дугмићи се убацују из Toolbox-а као Button, и то са следећим својствима:

За дугме Start log:



Односно за дугме Stop log:



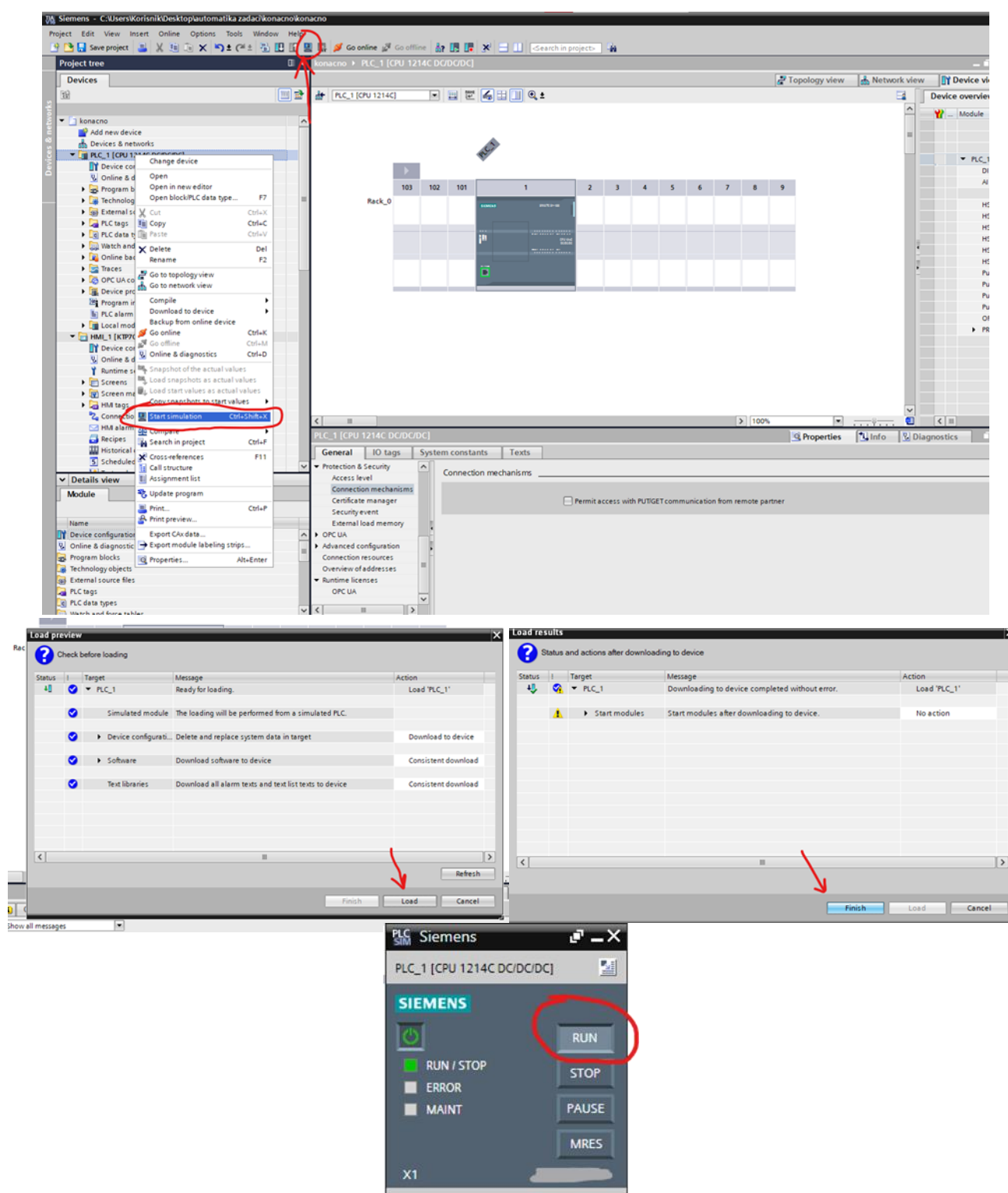
Добијени текстуални фајл ће се по дифолту чувати на адреси Local disk/[Storage Card SD/USB_X60.1]/[име лог фајла].

Када сматрамо да смо завршили едитовању HMI панела, можемо компајловати сав хардвер за сваки случај и, ако нема грешака наставити даље.

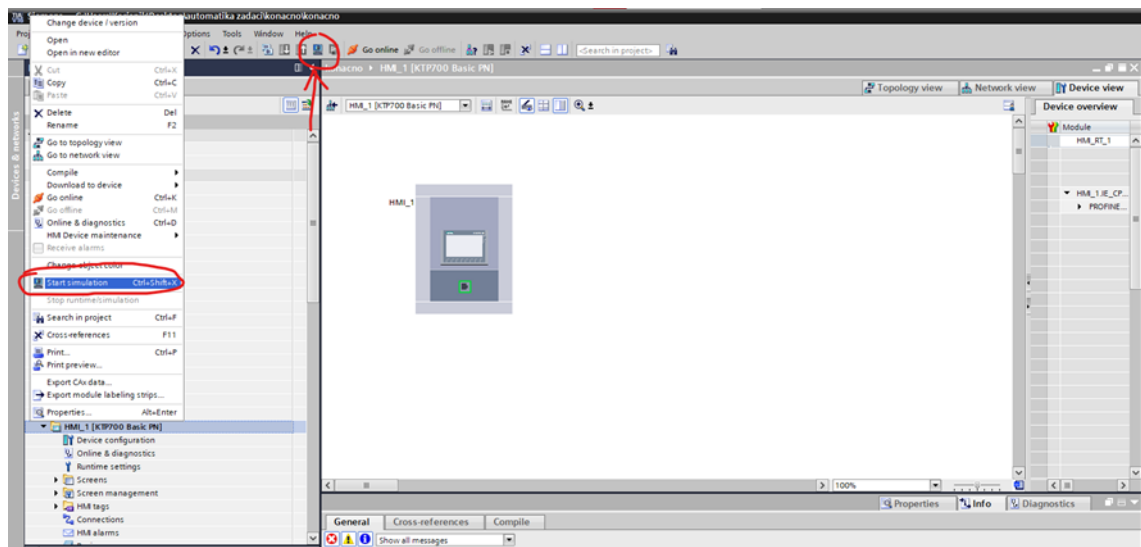
5. ПОКРЕТАЊЕ СИМУЛАЦИЈЕ И ДИСКУТОВАЊЕ РЕЗУЛТАТА

Након завршеног подешавања HMI панела, можемо покренути симулацију и погледати резултате.

Покретање симулације у TIA Portalu се врши у више корака. Прво је потребно да покренемо симулацију самог PLC дела, као на сликама:



Затим покрећемо симулацију HMI-а на исти начин као и PLC, односно:

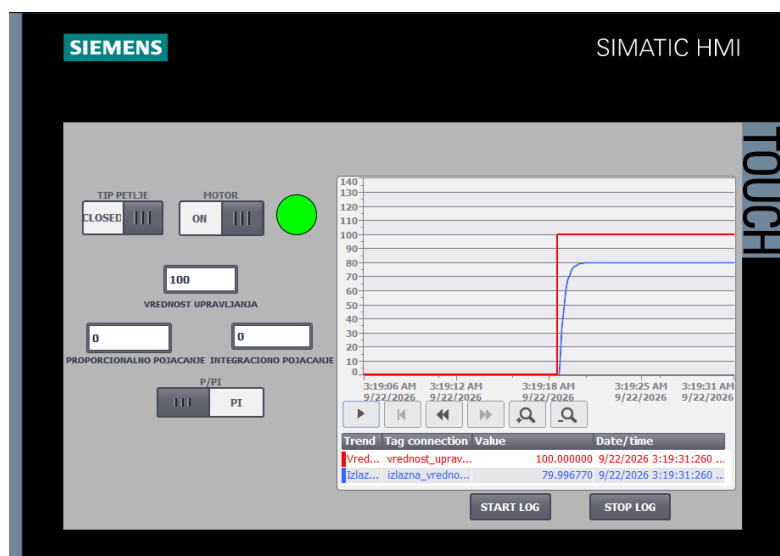


Када се заврши компајлирање и отвори симулирани HMI, спремни смо за симулације.

Како би смо упоредили управљање постигнуто П и ПИ контролером, спровешћемо више симулација, прво ћемо погледати графике одзива система у отвореној спрези, затим под утицајем П и ПИ контролера респективно. Упоредо са симулацијама које радимо овде, радимо симулације и у MATLAB-у и у Simulink-у, и упоређујемо резултате како би доказали тачност симулације.

5.1. Рад система у отвореној петљи

Како би смо анализирали одскочни сигнал који објекат произведе побудом на степ сигнал вредности 100, потребно је да прво на панели укључимо сам мотор, и затим задамо управљачки сигнал од 100. За овакве параметре, на графику се јавља следећа слика:

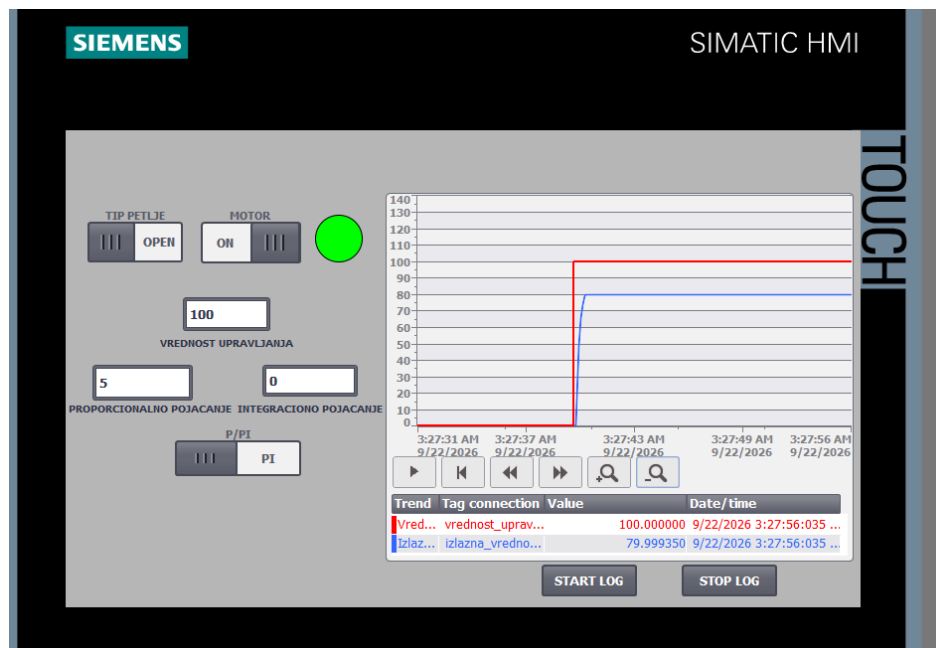


Са слике се види да за побудни сигнал од 100, објекат постиже излазну вредност од тек 80 посто, уз доста спор одговор система на побудни сигнал. Ово се у потпуности поклапа са нашим објектом из ЕПФЛ вежби који услед трења и губитка енергије постиже тек вредност од 80 посто од управљачког сигнала.

У овом кораку је такође погодно извршити логирање вредности тако што притиснемо дугме за старт пре укључивања мотора и стоп, након постизања стационарне вредности објекта. Добијени текстуални фајл ће касније бити разматран при анализи резултата.

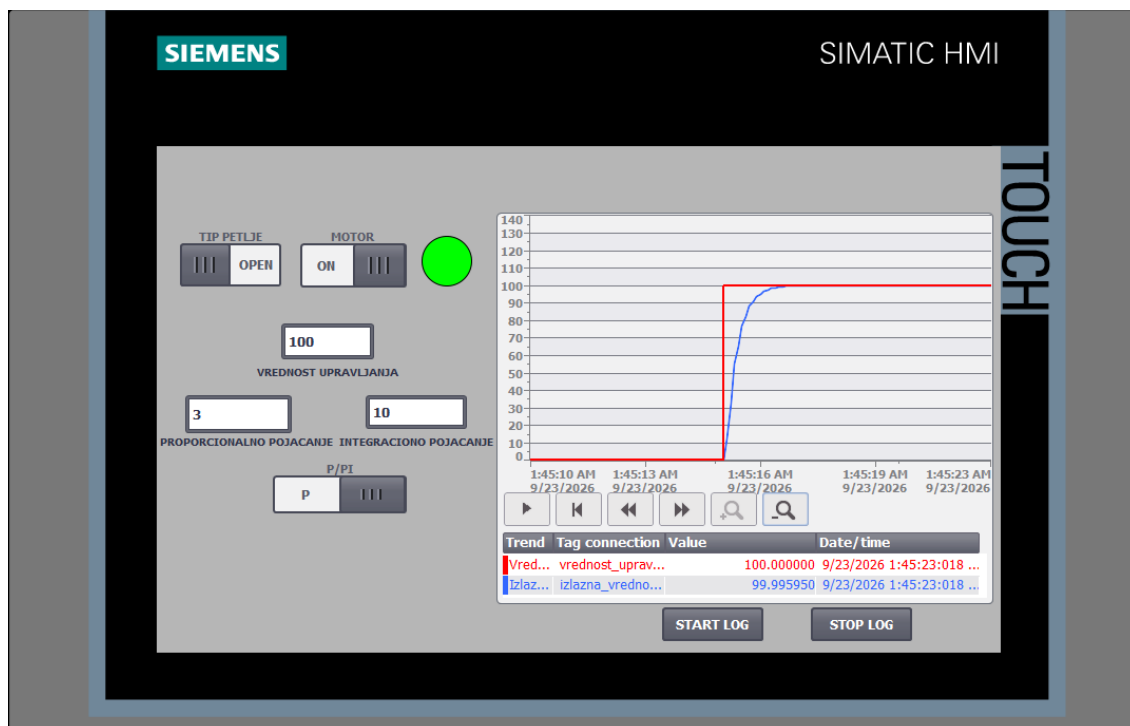
5.2. Рад система у затвореној повратној спрези

Ако сада затворимо повратну спрегу притиском на прекидач на панелу, потребно је да унесемо вредности одговарајућих појачања које користе контролери при управљању над објектом. Па тако уношењем пропорционалног појачања од 5, ми за исти побудни сигнал од 100, добијамо следећи график:

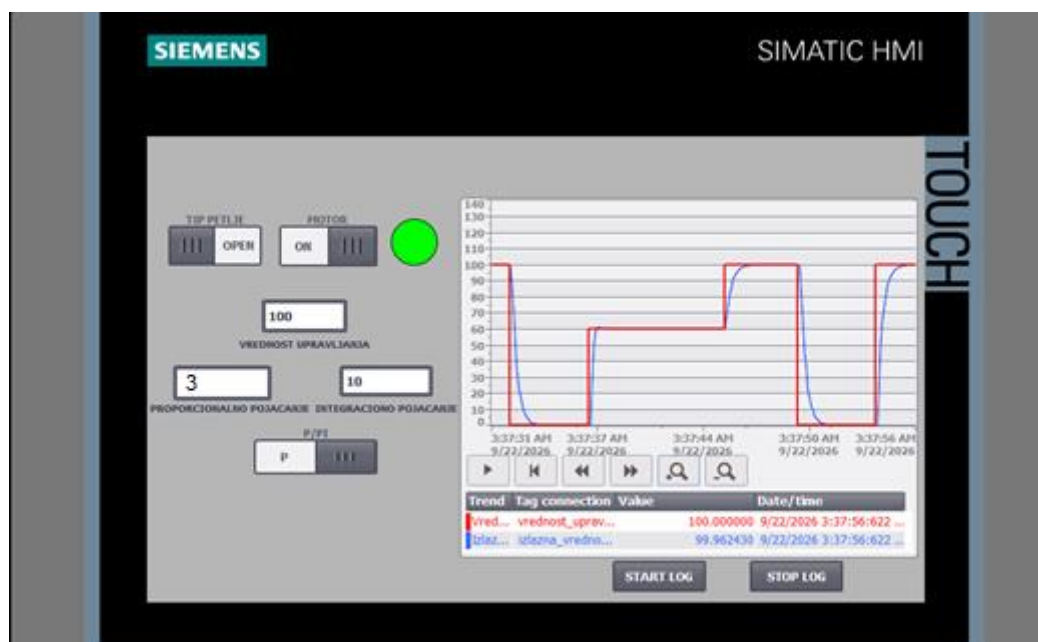


Пропорционално појачање 5 нам убрзава одзив система (што се види из тога да је плава линија ближа црвеној него на претходном графику), али и даље постоји грешка стационарног стања. Ако ми сада у овој конфигурацији задатка повећавамо јачину пропорционалног појачања на неке високе бројеве, приметимо да то доводи само до смањивања стабилности система, док се упоредно смањује и грешка стационарног стања, али она никад не постиже жељену вредност од 100, због непоседовања интеграционог члана при рачуну.

Ако пак пребацимо симулацију на задатак у затвореној повратној спрези под регулацијом ПИ контролера, и задамо вредност интеграционог појачања 10, добија се следећи график:



Пројектовани контролер у потпуности успева да елиминира грешку стационарног стања, уз задржавање брзине система које нам обезбеђује пропорционално дејство. Овај регулатор не само што елиминира грешку стационарног стања, већ се и врло добро стабилизује ако га пореметимо разним скоковитим променама вредности управљања, што се види са слике:



Наравно, вредности појачања регулатора се могу мењати произвољно, али за потребе задатка је потребно користити најефективнију комбинацију појачања, што је у овом случају $K_c = 3$ и $K_i = 10$.

6. Снимање и анализа временских одзива система (TIA Portal Trace)

За физичку промену алгоритма управљања и дигиталног модела објекта, кључно је да аквизицију података вршимо са стриктно детерминистичким периодом одабирања.

У ту сврху, унутар PLC програма искоришћен је циклични прекидни блок OB30, који смо направили и попунили у претходним поглављима, а чији је период извршавања конфигурисан на фиксних $T_s = 40$ [ms]. Све кључне дискретне једначине – дигитални модел објекта $G(s) = 0.8/(0.3s + 1)$ и дискретни PI регулатор са Anti-Windup заштитом, извршавају се искључиво унутар овог блока.

За снимање ових мерних сигнала, користимо интегрисани системски алат TIA Portal Trace. Интеграција и подешавање аквизиције спроведени су према следећим параметрима.

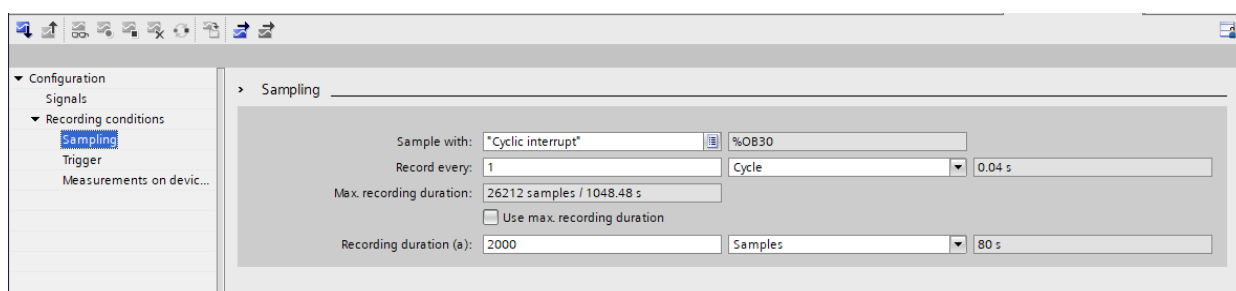
Вршимо синхронизовање са OB30 блоком, тако што тригер и “sampling” Trace алата директно повезујемо са цикличним прекидом OB30, чиме је обезбеђено да се сваки узорак снима на сваких 40 [ms].

Вршимо раздвајање од HMI слоја, HMI систем (Trend View и Historical Data) конфигурисан је за оператерски надзор са споријим периодом освежавања (100 [ms]) ради смањења оптерећења комуникације, док се сви подаци за детаљну анализу и графички приказ искључиво преузимају из TIA Portal Trace-а.

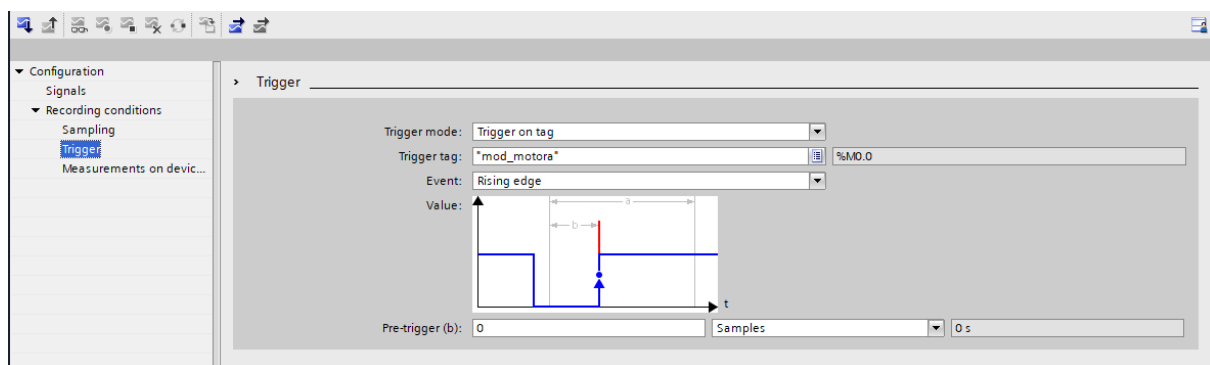
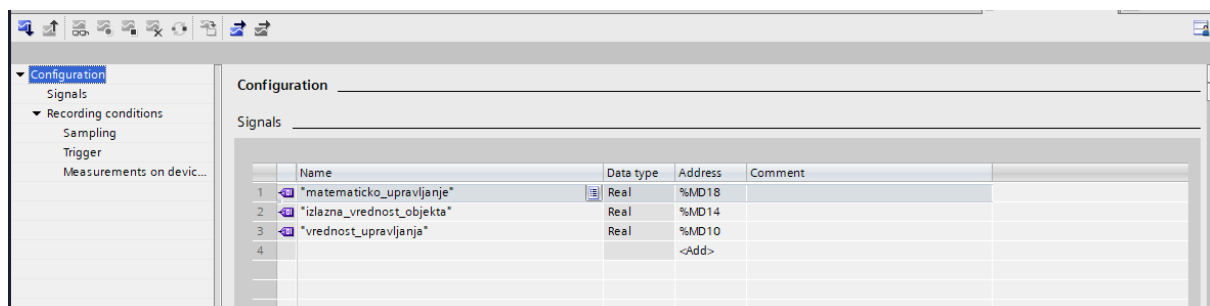
Током свих експеримената истовремено су праћена три кључна сигнала: vrednost_upravljanja, w[k] (односно референтни СТЕП сигнал), matematicko_upravljanje u[k] (излаз регулатора), izlazna_vrednost_objekta y[k] (вредност одзива дигиталног модела).

Експорт података вршимо тако што снимљене временске дијаграме извеземо у .csv/.txt форматима, што омогућава њихово директно учитавање и графичку обраду у MATLAB окружењу ради поређења са теоријском симулацијом.

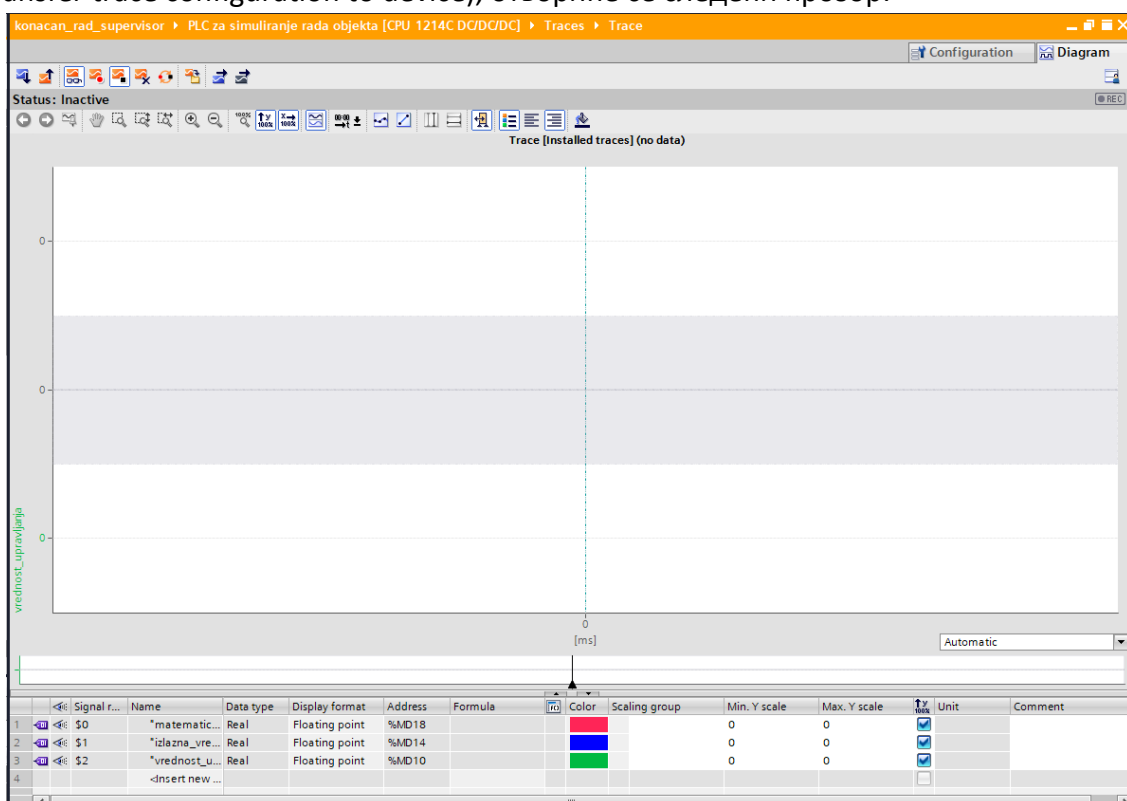
Прво додајемо нови Trace из Project tree-а са леве стране, кликом на Add new trace, са следећим параметрима:



Испитни задатак из Аутоматског управљања

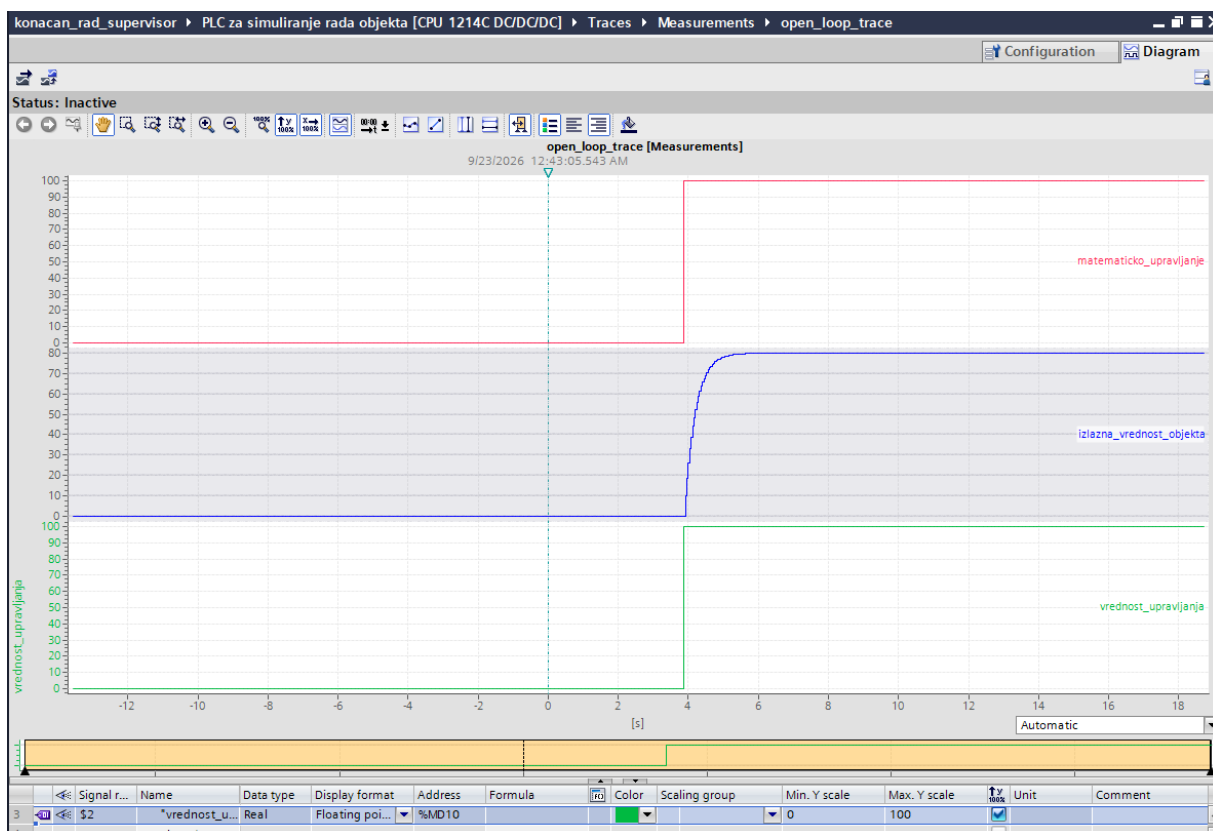


Када кликнемо дугме из горњег менија за трасфер дате конфигурације на наш систем (Transfer trace configuration to device), отвориће се следећи прозор:

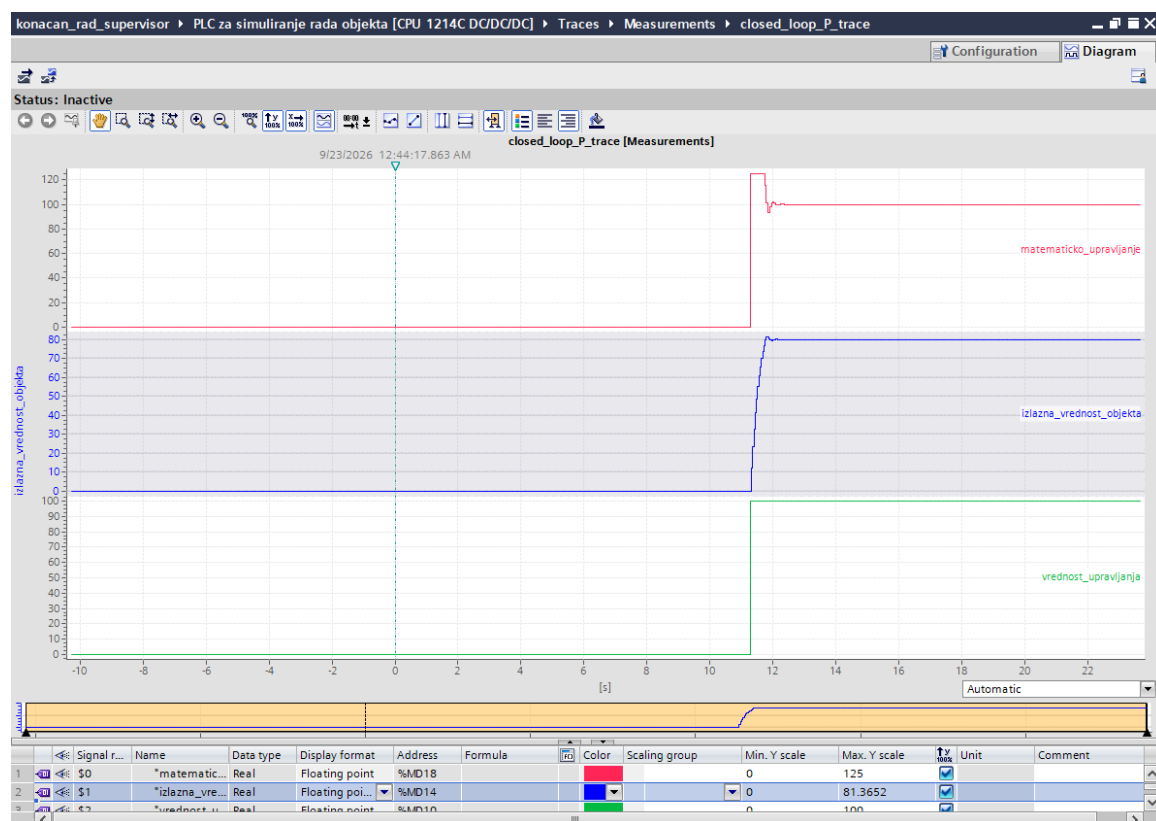


Напомена да уколико сам виртуелни ПЛЦ није активан, нећемо моћи да трансферујемо конфигурацију овако, док не упалимо ПЛЦ. Након овога, кликом на другме за почетак снимања отпочињемо снимање резултата, али тек од оног тренутка када упалимо сам мотор на HMI панелу. Тако, можемо направити „snapshot“-ове наших резултата и експортирати их касније.

У сврху испитивања исправности сваког дела овог пројекта, извршена су 4 мерења, са резултатима као на сликама:

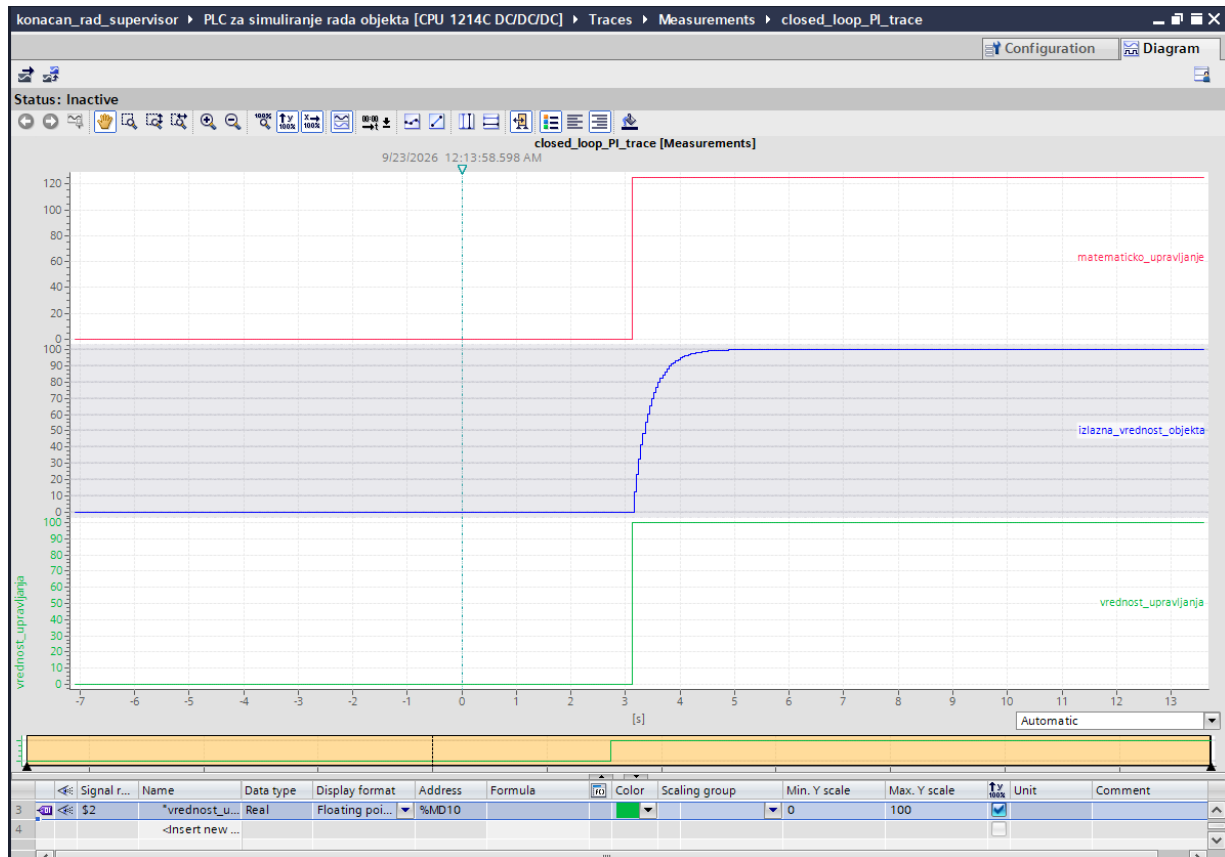


Слика 1: Снимак одзива система у отвореној вези на СТЕП сигнала вредности 100

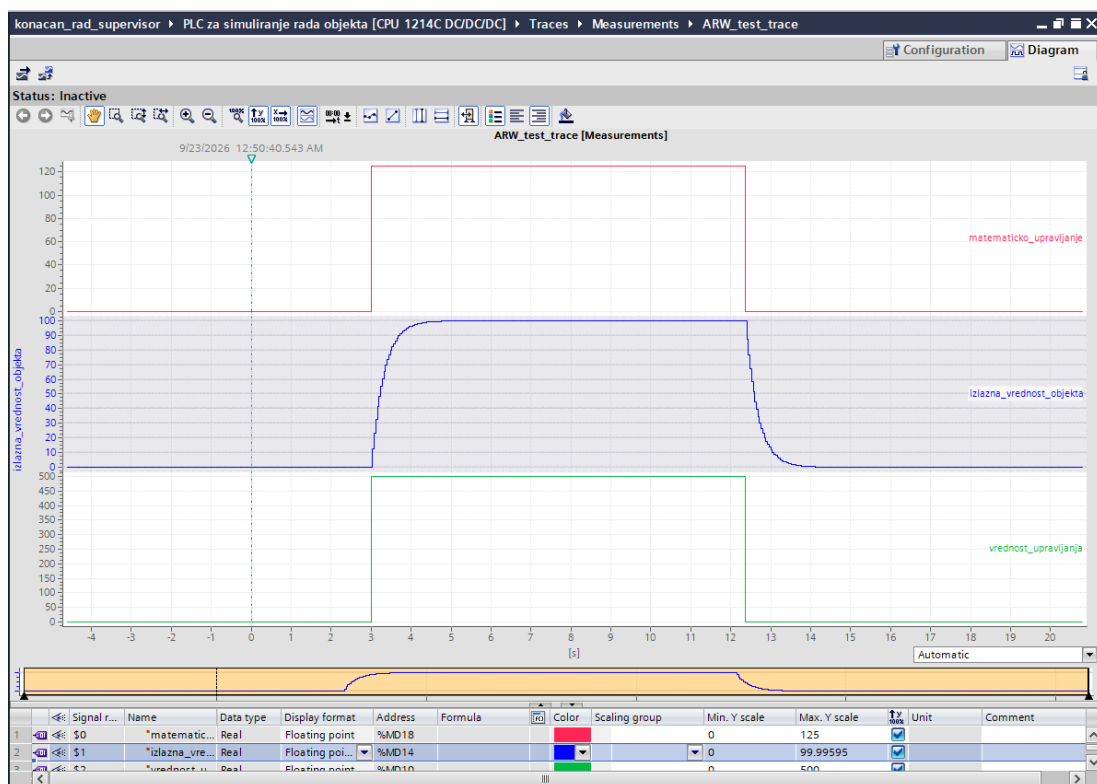


Слика 2: Снимак одзива система у затвореној повратној спрези под П регулатором ($w = 100$)

Испитни задатак из Аутоматског управљања



Слика 3: Снимак одзива система затвореној повратној спрези под ПИ регулатором ($w = 100$)



Слика 4: Снимак одзива система при задавању улазне вредности од 500, услед доказивања деловања ARW механизма, односно да не постоји препуњавање интегратора које успорава одзив система

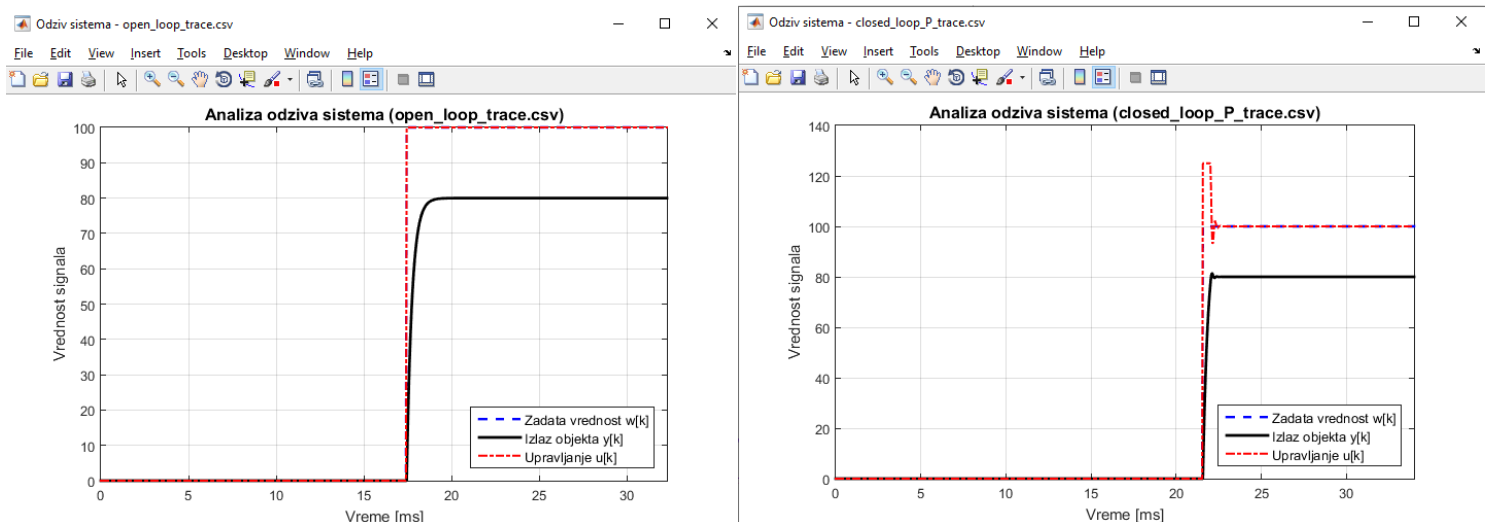
Ако добијене вредности сада екпортујемо ван TIA Portal-а, добићемо 4 .csv фајла, чије исцртавање на график може обавити следећа скрипта:

```

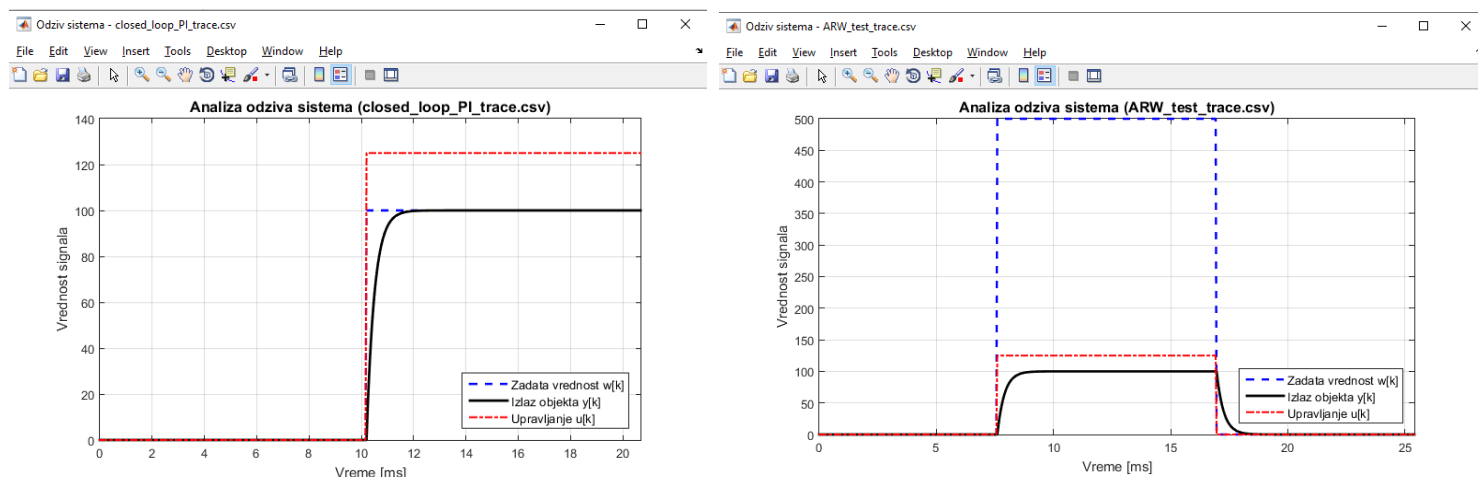
1 - clear all;
2 - clc;
3 - close all;
4
5 - [filename, pathname] = uigetfile({'*.csv;*.txt', 'Trace fajlovi (*.csv, *.txt); *.*', 'Svi fajlovi (*.*)'}, ...
6 -     'Odabir fajla');
7
8 - if isequal(filename, 0) || isequal(pathname, 0)
9 -     disp('otkazano');
10 -    return;
11 - end
12
13 - fullFilePath = fullfile(pathname, filename);
14
15 - try
16 -     rawData = importdata(fullFilePath);
17 -     if isstruct(rawData)
18 -         matrixData = rawData.data;
19 -     else
20 -         matrixData = rawData;
21 -     end
22 - catch
23 -     matrixData = readmatrix(fullFilePath);
24 - end
25
26 - if isempty(matrixData)
27 -     error('greska');
28 - end
29
30 - [rows, cols] = size(matrixData);
31
32 - Ts = 0.04;
33 - time = (0:rows-1) * Ts;
34
35 - u = matrixData(:, end-2);
36 - y = matrixData(:, end-1);
37 - w = matrixData(:, end);
38
39 - figure('Name', ['Odziv sistema - ' filename], 'NumberTitle', 'off', 'Color', [1 1 1]);
40
41 - plot(time, w, 'b--', 'LineWidth', 1.8); hold on;
42 - plot(time, y, 'k-', 'LineWidth', 2.0);
43 - plot(time, u, 'r-.', 'LineWidth', 1.5);
44
45 - grid on;
46 - title(['Analiza odziva sistema (' filename ')'], 'Interpreter', 'none', 'FontSize', 12);
47 - xlabel('Vreme [s]', 'FontSize', 11);
48 - ylabel('Vrednost signala', 'FontSize', 11);
49
50 - legend(['Zadata vrednost w[k]', 'Izlaz objekta y[k]', 'Upravljanje u[k]'], ...
51 -     'Location', 'southeast', 'FontSize', 10);
52
53 - xlim([0 max(time)]);

```

Покретањем скрипте и бирањем одговарајућих фајлова, добијамо следеће дијаграме:



Испитни задатак из Аутоматског управљања



Сада, ако извршимо појединачну анализу ових графика закључујемо следеће:

На дијаграму одзива за отворену спрегу (слика са `open_loop_trace.csv`) приказан је одзив дигиталног модела објекта на директну одскочну побуду управљања амплитуде $u[k] = 100$. Са дијаграма се јасно види да излаз објекта $y[k]$ монотонно расте све док не достигне стационарну вредност од 80. Овакав одзив у потпуности потврђује постављени математички модел објекта првог реда $G(s) = 0.8/(0.3s + 1)$, где статичко појачање износи $K = y_{\infty}/u_{\infty} = 80/100 = 0.8$. Временски ток одзива прати теоријску временску константу од $T = 0.3$ [s], а одсуство било каквог кашњења или прескока на излазу потврђује да је дискретизација са периодом од $T_s = 40$ [ms] унутар блока ОВ30 спроведена и физички и математички тачно.

На дијаграму затворене петље регулисане П регулатором (`closed_loop_P_trace.csv`) анализира се понашање система у затвореној спрези под дејством пропорционалног регулатора при одскочној промени задате вредности са $w[k] = 0$ на $w[k] = 100$. У самом тренутку скока (t приближно једнако 22 [ms]) долази до тренутног пика односно прескока на управљачком сигналу $u[k]$ до вредности од 126.6, након чега се управљање брзо стабилизује на вредност од 100. Овај почетни прескок се јавља искључиво на управљачком сигналу $u[k]$ и представља директну последицу пропорционалног дејства ($u[k] = K_p * e[k]$) на тренутну максималну грешку система која у првом узорку износи $e[0] = 100 - 0 = 100$. Како излаз објекта $y[k]$ почиње да расте, грешка се смањује, што узрокује пад управљачког сигнала на стационарну вредност. Сам излаз објекта $y[k]$ нема велики прескок већ скоро монотонно достиже стационарно стање од $y_{\infty} = 80$, чиме се јасно демонстрира карактеристична преостала стационарна грешка ($e_{st} = 20$) која је неизбежна код чисте П регулације објекта са појачањем $K = 0.8$.

На дијаграму одзива са ПИ регулатором (`closed_loop_PI_trace.csv`) приказано је управљање у затвореној спрези где је додато интегрално дејство ради елиминације стационарне грешке. Приликом одскочне промени задате вредности на $w[k] = 100$, управљачки сигнал $u[k]$ снажно реагује и у почетном тренутку, и достиже вредност од 125. За разлику од П регулатора, континуално дејство интегратора омогућава да се управљање одржи на нивоу од 125, што као директну последицу има довођење излаза објекта $y[k]$ тачно на задану вредност од 100, пошто је $125 * 0.8 = 100$. Излаз објекта

показује гладак аperiodски прелазни процес без прескока и са нултом стационарном грешком и одзиву ($e_{st} = 0$), чиме су у потпуности испуњени сви квалитативни захтеви постављени пред наш ПИ регулатор.

На дијаграму ARW теста (ARW_test_trace.csv) приказана је практична верификација заштитног Anti-Windup механизма заснованог на “clamping” методи. Задана вредност је постављена на високих $w[k] = 500$, док је излаз регулатора физички ограничен сатурацијом на $u_{max} = 100$, услед чега излаз објекта достиже свој максимум од $y[k] = 100$. Током периода трајања сатурације, грешка система остаје велика и позитивна ($e[k] = 400$), али Anti-Windup алгоритам замрзава даље акумулирање интегралне суме. У тренутку када задана вредност нагло падне на $w[k] = 0$, управљачки сигнал $u[k]$ и излаз објекта $y[k]$ моментално крећу да опадају без икаквог временског кашњења или “мртвог хода”. Ова брза реакција директан је доказ да интегратор није претрпео “препуњавање” односно “windup” појаву, чиме је исправност имплементиране Anti-Windup логике у SCL коду практично и графички потврђена.

7. ПОРЕЂЕЊЕ РЕЗУЛТАТА: MATLAB/SIMULINK VS. PLC (TIA PORTAL)

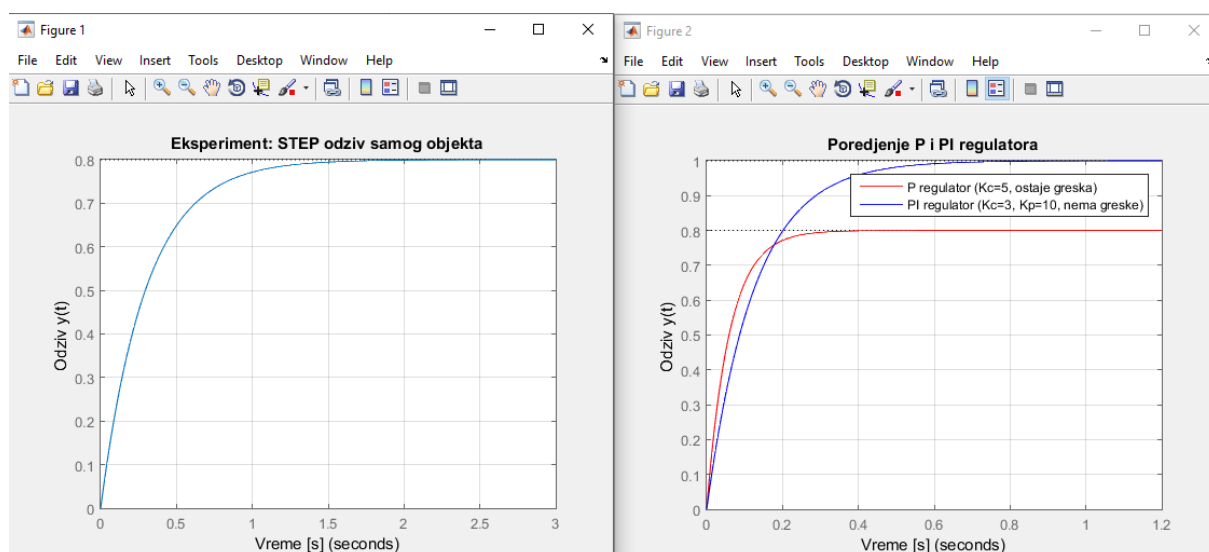
7.1. Симулација у MATLAB-у

Да би се извршила верификација тачности употребљеног SCL кода и дискретног модела у TIA Portal-у, спроведено је поређење добијених одзива са теоријским симулационим моделом изграђеним у програму MATLAB / Simulink.

У MATLAB-у је коришћена следећа скрипта како би се добили потребни графици:

```
Kp = 0.8;
T = 0.3;
Ts = 0.04;
G = tf(Kp, [T 1]);
figure(1);
step(G);
title('Eksperiment: STEP odziv samog objekta');
xlabel('Vreme [s]'); ylabel('Odziv y(t)');
grid on;
Kc_p = 5.0;
W_p = feedback(Kc_p * G, 1);
Kc_pi = 3.0;
Ti = 0.3;
C_pi = tf(Kc_pi * [Ti 1], [Ti 0]);
W_pi = feedback(C_pi * G, 1);
figure(2);
step(W_p, 'r', W_pi, 'b');
title('Poredjenje P i PI regulatora');
legend('P regulator (Kc=5, ostaje greska)', 'PI regulator (Kc=3, Ki=10, nema greske)');
xlabel('Vreme [s]'); ylabel('Odziv y(t)');
grid on;
```

Када покренемо скрипту добијамо следећа два дијаграма:



- Одзив самог објекта у отвореној петљи (figure(1))

Анализира се одзив објекта $G(s) = 0.8/(0.3s + 1)$ на одскачну побуду без присутности регулатора. Резултат који добијамо јесте да излаз $y(t)$ експоненцијално расте, и у стационарном стању достиже вредност једнаку статичком појачању објекта $y_{st} = K_p = 0.8$ (80%).

- Поређење П и ПИ регулатора у затвореној петљи (figure(2))

За П регулатор (црвена линија W_p), при $K_{c_p} = 5.0$, преносна функција затвореног кола износи:

$$W_p(s) = \frac{5 \cdot G(s)}{1 + 5 \cdot G(s)} = \frac{5 \cdot \frac{0.8}{0.3s+1}}{1 + 5 \cdot \frac{0.8}{0.3s+1}} = \frac{4}{0.3s + 5} = \frac{0.8}{0.06s + 1}$$

Стационарна вредност је $y_{st} = 0.8$ (80%), што значи да у затвореном колу остаје статичка грешка од 20% ($est = 1 - 0.8 = 0.2$). Одзив је бржи у односу на отворену петљу ($\tau_{zk} = 0.05$ s), али грешка остаје.

За ПИ регулатор (плава линија W_{pi}): Применом ПИ регулатора са $K_{c_pi} = 3.0$ и $T_i = 0.3$ s остварује се поништавање доминантног пола објекта ($T_i = T = 0.3$ s). Преносна функција регулатора је:

$$C_{pi}(s) = 3 \cdot \frac{0.3s + 1}{0.3s}$$

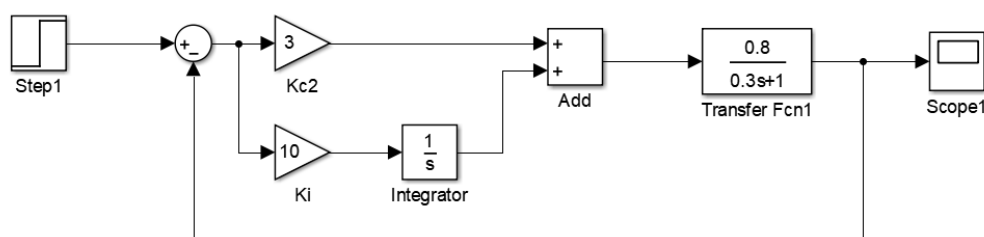
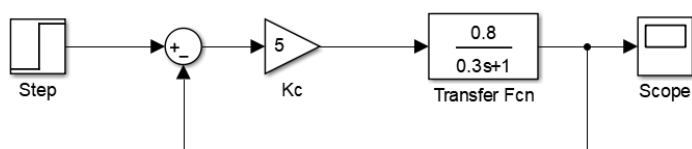
Преносна функција затвореног кола постаје:

$$W_{pi}(s) = \frac{C_{pi}(s) \cdot G(s)}{1 + C_{pi}(s) \cdot G(s)} = \frac{\frac{2.4}{0.3s}}{1 + \frac{2.4}{0.3s}} = \frac{2.4}{0.3s + 2.4} = \frac{1}{0.125s + 1}$$

Излаз $y(t)$ асимптотски достиже тачно 1.0 (100%), чиме је статичка грешка потпуно елиминисана ($est = 0$), без појаве прекорачења (препуњавања интегратора).

7.2. Симулација у SIMULINK-у

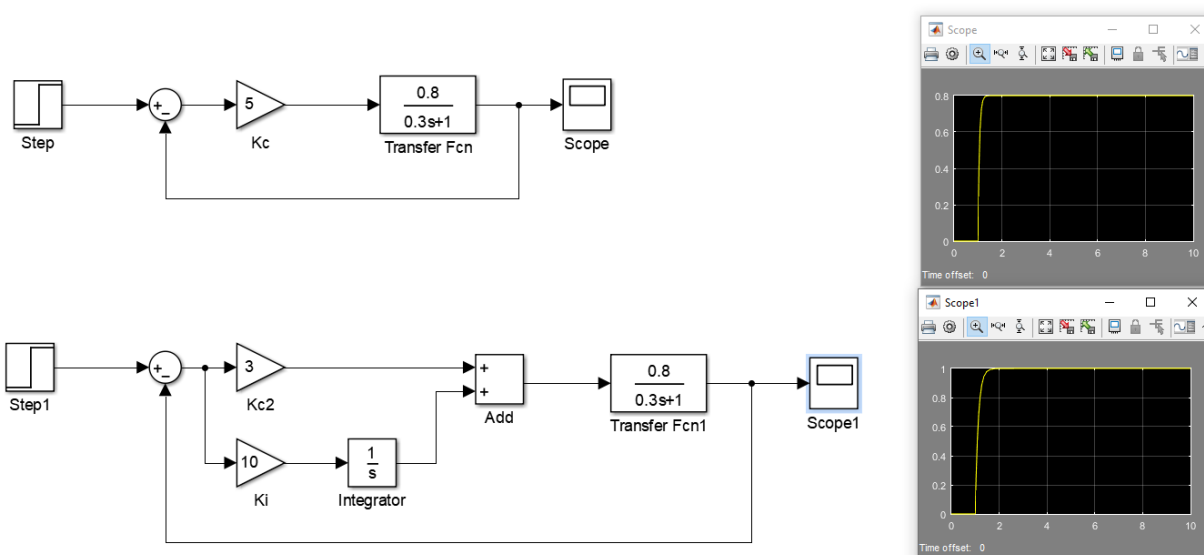
Модел се састоји од два одвојена подсистема који паралелно анализирају одзив објекта $G(s) = 0.8/(0.25s + 1)$ на јединичну одскачну побуду ($Step = 1.0$).



- Горњи подсистем – Чист П регулатор ($K_c = 5$):
 - Step: Улазни сигнал задате вредности $w(t) = 1.0$
 - Sum блок (+-): Формира сигнал грешке $e(t) = w(t) - y(t)$.
 - Kc (Gain блок = 5): Пропорционално појачање регулатора.
 - Transfer Fcn ($0.8/(0.3s+1)$): Континуални модел објекта управљања.
 - Scope: Блок за снимање и приказ одзива излазне величине $y(t)$.
- Доњи подсистем – Паралелни ПИ регулатор ($K_{c2} = 3$, $K_i = 10$):
 - Step1: Улазни сигнал задате вредности $w(t) = 1.0$
 - Sum блок (+-): Формира сигнал грешке $e(t) = w(t) - y(t)$
 - Пропорционална грана ($K_{c2} = 3$): Појачава сигнал грешке са коефицијентом $K_c = 3$

4. Интегрална грана ($K_i = 10$ и Integrator $1/s$): Формира интегрално дејство $K_i \int e(t) dt$, где је континуално интегрално појачање $K_i = 10.0 \text{ s}^{-1}$ (што директно одговара дискретном кораку $K_{i_step} = 0.4$ за $T = 0.04 \text{ s}$ јер је $K_i * T = 10 * 0.04 = 0.4$)
5. Add блок: Сабира пропорционалну и интегралну компоненту у укупни управљачки сигнал $u(t) = P(t) + I(t)$
6. Transfer Fcn1 ($0.8/(0.3s+1)$): Модел објекта
7. Scope1: Блок за праћење одзива ПИ регулатора.

Дуплим кликом на два Scope-а добијамо следеће дијаграме:



Приказ са Scope-а (за П регулатор) потврђује горенаведене ставка, да систем са пропорционалним појачањем $K_c = 5$ достиже стационарну вредност $y_{st} = 0.8$. Грешка од 20% остаје трајно присутна јер управљачки сигнал опада како се излаз приближава задатој вредности, па нема елемента који би додатно погурао овај излаз до 1.0.

Приказ са Scope-а 1 (за ПИ регулатор) нам потврђује да интегратор у доњој грани успешно акумулира грешку кроз време све док је $e(t)$ различито од 0. Због тога излаз асимптотски али брзо достиже пуну вредност $y_{st} = 1.0$, а статичка грешка се потпуно елиминише ($e_{st} = 0$).

7.3. Читање log текстуалног фајла у MATLAB-у и исцртавање графика

Након што лоцирамо текстуални фајл на систему оператера(по дифолту то је фајл који креира сама TIA Portal,на локацији C:\Storage Card SD\Log\Loging_file), потребно је да га убацимо у следећу MATLAB скрипту за читање података:

Испитни задатак из Аутоматског управљања

```

1
2 - [filename, filepath] = uigetfile('*.txt', 'Izaberite log fajl');
3
4 - if isequal(filename, 0)
5 -     disp('Izbor fajla je otkazan.');
```

```

6 -     return;
7 - end
8
9 - fullpath = fullfile(filepath, filename);
10
11
12 - fid = fopen(fullpath, 'r', 'n', 'UTF-16LE');
```

```

13 - if fid == -1
14 -     fid = fopen(fullpath, 'r');
```

```

15 - end
16
17
18 - raw = textscan(fid, '%q %q %f %f %f', 'HeaderLines', 1);
19 - fclose(fid);
20
21 - varNames = cellstr(raw{1});
22 - values = raw{3};
23 - time_ms = raw{5};
24
25
26 - validIdx = ~strcmp(varNames, '$', 1);
27 - varNames = varNames(validIdx);
28 - values = values(validIdx);
29 - time_ms = time_ms(validIdx);
30
31
32 - t_start = min(time_ms);
33 - t_all = (time_ms - t_start) / 1000;
```

```

34
35
36 - uniqueVars = unique(varNames);
37
38 - figure('Name', ['Prikaz signala: ' filename], 'NumberTitle', 'off');
```

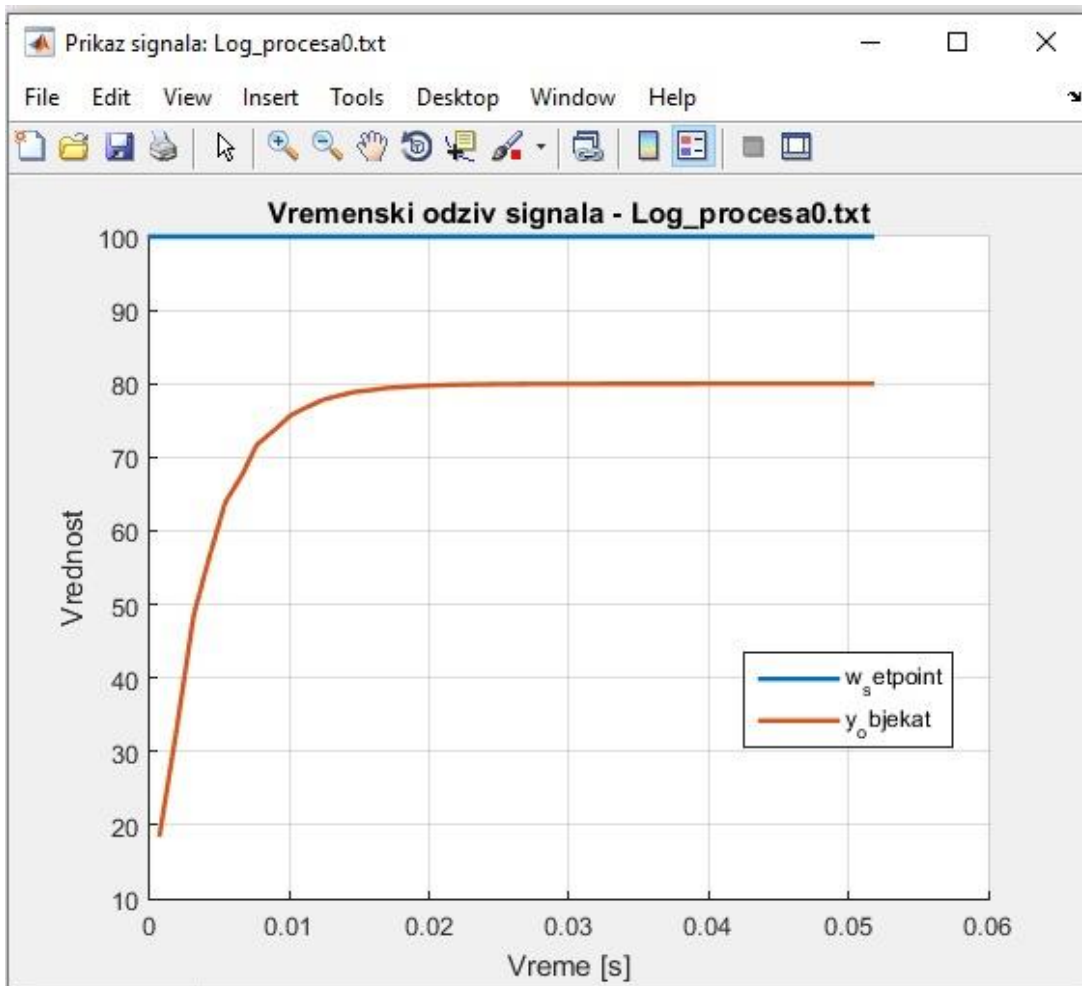
```

39 - hold on;
40
41 - for i = 1:length(uniqueVars)
42 -     currentVar = uniqueVars{i};
43 -     idx = strcmp(varNames, currentVar);
44
45 -     t_var = t_all(idx);
46 -     y_var = values(idx);
47
48
49 -     if length(t_var) == 1
50 -         t_var = [t_var; max(t_all)];
51 -         y_var = [y_var; y_var(1)];
52 -     end
53
54
55 -     plot(t_var, y_var, 'LineWidth', 1.8, 'DisplayName', currentVar);
56 - end
57
58 - hold off;
59 - grid on;
60 - xlabel('Vreme [s]');
61 - ylabel('Vrednost');
```

```

62 - title(['Vremenski odziv signala - ', strrep(filename, '_', '\_')]);
63 - legend('Location', 'best');
```

Уколико смо добро снимили податке из симулације, на графику који изађе појавиће се нешто овако:



Будући да посматрамо експеримент са отвореном спрегом, резултати се поклапају у потпуности са теоријски добијеним вредностима.

Са овим смо завршили проверу резултата.

Сви фајлови, скрипте и зип-документи окачени су на Github репозиторијуму:

https://github.com/pointblank-100/Automatsko_upravljanje.git

8. Закључак

У овом раду успешно је пројектован, имплементиран и анализиран дискретни систем аутоматског управљања за систем првог реда преносне функције $G(s) = 0.8/(0.3s + 1)$. Симулација и валидација алгоритма извршене су коришћењем програмибилног логичког контролера Siemens S7-1214 DC/DC/DC у окружењу TIA Portal, као и рачунарских пакета MATLAB и Simulink. На основу спроведеног истраживања и експерименталних резултата, могу се дефинисати следећи кључни закључци.

Дискретизација континуалног модела Ојлеровом методом и његова имплементација у SCL језику унутар цикличног прекидног блока OB30 ($T = 40 \text{ ms}$) показала је велику прецизност. Одзиви добијени на PLC-у у потпуности одговарају теоријским одзивима из MATLAB/Simulink симулација.

Примена чистог пропорционалног регулатора ($K_c = 5.0$) потврдила је основни теоријски недостатак овог типа управљања — карактеристичну статичку грешку (20.0%). Иако повећање појачања K_c убрзава одзив и смањује грешку, она се не може потпуно елиминисати без интегралног дејства. Увођењем интегралне компоненте ($K_{c_PI} = 5$, $K_i = 10$) статичка грешка је потпуно елиминисана ($e_{st} = 0\%$), а излазна величина је асимптотски достигла задату вредност без појава прекорачења или осцилација.

Имплементација ARW алгоритма са засићењем интегратора унутар SCL кода спречила је прекомерно нагомилавање интегралне суме током великих одступања, чиме је обезбеђена стабилност и адекватан прелазни процес при симулацији реалних ограничења излаза. Развијени HMI интерфејс (Root screen) са Trend View графиком омогућио је интуитивно праћење процеса у реалном времену, дијагностику и динамичку измену параметара помоћу параметарских поља на симулираном панелу. Овим је потврђено да креирана управљачка логика за Siemens S7-1200 представља поуздану, математички тачну и практично применљиву основу за управљање реалним индустријским процесима првог реда.